

Market Game 2027 — comprendre le code étape par étape

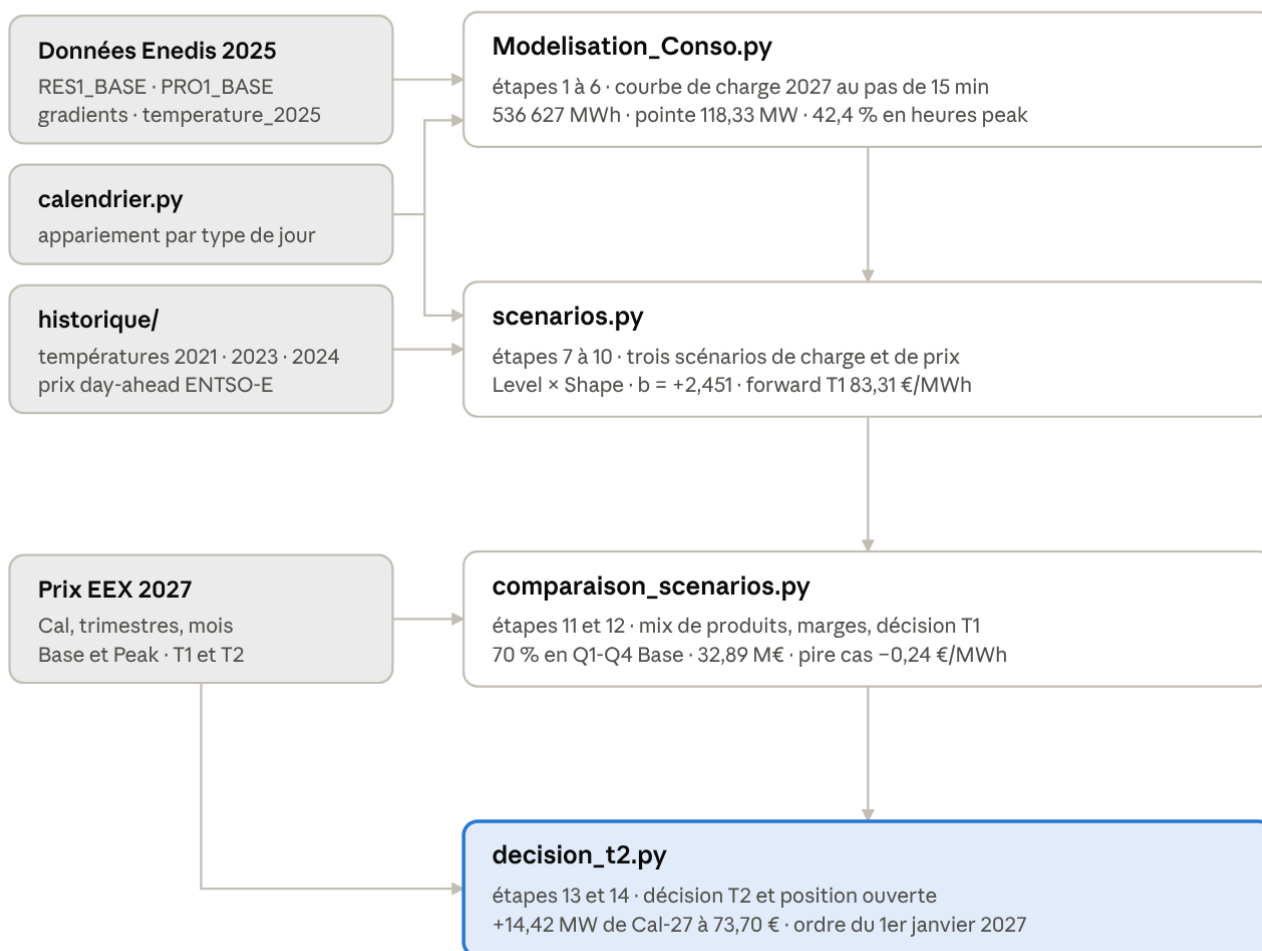
2026-10-03 · Jean

Le portefeuille 2027 pèse 536 627 MWh à température normale. Ce document suit, étape par étape, le chemin que parcourent les cinq scripts Python : du coefficient de profil Enedis jusqu'à l'ordre day-ahead du 1er janvier 2027 — la théorie du cours d'un côté, le bout de code qui l'applique de l'autre.

Le circuit

Des profils Enedis à l'ordre day-ahead du 1er janvier 2027

Cinq scripts, quatorze étapes, trois jeux de données



circuit du modèle · 3 sources, 4 scripts, 14 étapes

Chaque script reconstruit tout depuis les CSV de base : aucun ne lit les sorties d'un autre. C'est ce qui permet de relancer `decision_t2.py` seul après avoir changé un paramètre, et c'est aussi pourquoi les deux derniers mettent quelques minutes à tourner.

Les chiffres clés

Tout ce qu'on peut vous demander de citer de mémoire, regroupé ici.

Paramètres imposés (§2.2, identiques pour toutes les équipes)

Prix de vente, composante énergie	Marge cible	Plancher de risque	Prime de risque forward
95 €/MWh	10 €/MWh	5 €/MWh	3 %

Le volume

	Valeur
Énergie 2027, température normale	536 627 MWh = le 100 % de tous les ratios
Pointe	118,33 MW le 07/01 à 18h45
Répartition RES1 / PRO1	76,7 % / 23,3 %
Trimestres (MWh)	Q1 152 934 · Q2 118 293 · Q3 115 024 · Q4 150 376
Peak / off-peak	42,4 % / 57,6 %, ratio de forme 1,32
Foisonnement	8,5 % (136,98 MW de pointes cumulées contre 125,40 pour le portefeuille)
Thermosensibilité	1,96 MW/°C en moyenne de chauffe
Volume révisé au T2	501 962 MWh (−6,46 %)

Le prix

	Valeur
Forward Cal-27 Base au T1 (22/09/2026)	83,31 €/MWh
Forward Cal-27 Base au T2 (01/12/2026)	73,70 €/MWh (−11,5 %)
Pente prix-volume mutualisée	b = +2,451 ($r^2 = 0,243$, $t = +12,1$, $n = 454$)
Levels des scénarios	2021 80,72 · 2023 78,00 · 2024 77,46 · choc 96,87
Surcoût prix-volume	+4,18 à +6,65 €/MWh selon le scénario
Alpha retenu	0,50 — mais aucun alpha de [0 ; 1] n'atteignait 73,70
Bande de plausibilité CRE à 1 sigma	7,30 €/MWh, soit [76,01 ; 90,61]

La couverture

	Valeur
Mix T1 retenu (70 %, strip trimestriel Base)	Q1 49,58 MW · Q2 37,91 · Q3 36,47 · Q4 47,65
Coût de ce mix	32,89 M€, soit 87,55 €/MWh
Coût des trois mix comparés	cal 83,31 · trimestres 87,55 · trimestres+peak 89,11 €/MWh
Stratégie retenue au T1	70/20/10 — espérance 4,51, pire cas −0,24
Décision du Conseil	aucune stratégie ne tient le plancher de 5 €/MWh
Coût de la protection	0,39 €/MWh d'espérance cédé par €/MWh de pire cas gagné

Le T2 et le 1er janvier

	Valeur
Ratio réellement atteint après le T1	74,8 % de la nouvelle prévision (plan : 70 %)
Mark-to-market de la position T1	−3,92 M€ , soit −10,43 €/MWh couvert
Probabilités révisées	2021 28,1 % · 2023 34,2 % · 2024 35,4 % · choc 2,3 %
Achat au T2	14,42 MW de Cal-27 Base à 73,70 = 126 323 MWh, 9,31 M€
Couverture finale	100 % — espérance 7,14, pire cas 5,51 (admissible)
01/01/2027	couverture plate 64,01 MW, charge de 42,21 à 98,71 MW
Position ouverte du jour	261,6 MWh à acheter , 150,4 MWh à revendre

Le T3 (31/12/2026)

	Valeur
Volume 2027 révisé	512 820 MWh (191 531 RES1 + 4 745 PRO1)
Forward Cal-27 Base	78,37 €/MWh (+6,3 % depuis le T2)
Position détenue	501 942 MWh à 84,06 €/MWh payés
Mark-to-market	-1,35 M€ , dont +590 k€ sur l'achat du T2
Ratio global	97,9 % — mais Q1 à 94,6 % et Q3 à 102,2 %
Rééquilibrage	acheter Q1 +3,68 MW et Q4 +2,98 MW, vendre Q2 et Q3 → +1,40 M€
Centrale solaire	80 MWc , 110,5 MWh le 1er janvier
01/01/2027, température prévue	+0,44 °C contre 4,79 °C de normale
Charge du jour	1 849,5 MWh (1 682,1 à température normale)
Heure 14 h - 15 h	charge 81,89 · couverture 64,00 · solaire 14,00 → short 3,89 MW

Où se trouve quoi

Quatorze étapes réparties sur cinq fichiers, dont un seul — `calendrier.py` — ne se lance jamais directement.

#	Étape	Fichier	Fonction ou constante	Doc du prof
1	Le portefeuille et les paramètres imposés	<code>Modelisation_Conso.py</code>	<code>PARAMS</code> l.11, <code>REVISION_T2</code> l.15	\$2.2
2	Du coefficient au mégawatt : le FUD	<code>Modelisation_Conso.py</code>	<code>fud()</code> l.30	\$3.1
3	Charger et aligner les données	<code>Modelisation_Conso.py</code>	<code>charger_*</code> (), <code>construire_base()</code> l.77	\$3.1
4	Le coefficient météo CM	<code>Modelisation_Conso.py</code>	<code>coefficient_meteo()</code> l.84	\$3.1
5	Le calcul de la charge	<code>Modelisation_Conso.py</code>	<code>calculer_puissances()</code> l.93	\$3.1
6	Projeter sur le calendrier 2027	<code>calendrier.py</code>	<code>apparier_jours()</code> l.31, <code>projeter()</code> l.61	\$3.1
7	Rejouer une météo historique	<code>scenarios.py</code>	<code>injecter_temperature()</code> l.127	\$3.2
8	La régression prix-volume	<code>scenarios.py</code>	<code>anomalies()</code> l.142, <code>pente_prix_volume()</code> l.156	\$3.2
9	Level × Shape	<code>scenarios.py</code>	<code>scenario_spot()</code> l.169, <code>construire_scenario()</code> l.176	\$3.2
10	La méthode alpha	<code>scenarios.py</code>	<code>forward_t2_anticipe()</code> l.40, <code>sigma_t2()</code> l.45	\$3.3
11	Le mix de produits EEX	<code>comparaison_scenarios.py</code>	<code>fenetre()</code> l.37, <code>construire_mix()</code> l.68	\$3.4
12	Les marges et le choix	<code>comparaison_scenarios.py</code>	<code>marge()</code> l.160, <code>choisir()</code> l.205	\$3.5
13	Le travail au T2	<code>decision_t2.py</code>	<code>probabilites_revisees()</code> l.20, <code>marge_t2()</code> l.50	\$4.1 à 4.4
14	La position ouverte du 1er janvier	<code>decision_t2.py</code>	<code>fin de main()</code> , <code>tracer()</code> l.212	\$5

Ordre d'exécution : `Modelisation_Conso.py`, puis `comparaison_scenarios.py`, puis `decision_t2.py`. `scenarios.py` est facultatif (il détaille un seul scénario), `calendrier.py` est une bibliothèque. Les deux derniers scripts sont indépendants : chacun reconstruit tout depuis les CSV de base, aucun ne lit les sorties de l'autre.

Pour tout lancer d'un coup : `python lancer.py --figures` enchaîne les cinq scripts dans l'ordre et écrit tout dans `sorties/`, figures comprises — une trentaine de secondes. Le dossier est rangé en trois : `donnees/` pour les fichiers du prof (le code n'y écrit jamais), `sorties/` pour tout ce que le code produit (effaçable sans risque), `rapport/` pour ce document.

Les figures : chaque script se termine par un appel à `tracer()` et ouvre ses figures matplotlib à l'écran quand on le lance. Les sept figures reproduites dans ce document sortent telles quelles de ces appels, sans retouche. `figures.py`, dans le même dossier, rejoue les cinq scripts et enregistre les sept PNG dans `sorties/figures/` — il n'ajoute rien, il remplace seulement `plt.show()` par un `savefig()`.

Je cherche quoi, j'ouvre quel fichier

Tout ce qui se règle est **en tête de fichier**, avant la première fonction. Jamais besoin de descendre dans le corps du code pour changer un paramètre.

Ce que je veux voir ou changer	Fichier	Ligne
Année cible	Modelisation_Conso.py	9
FUD : clients, kVA, Ø	Modelisation_Conso.py — PARAMS	11
Les trois jalons commerciaux	Modelisation_Conso.py — CLIENTS	15
Seuil de chauffe 15 °C	Modelisation_Conso.py	22
Noms des fichiers CSV	Modelisation_Conso.py — FICHIERS	26
Les 4 cas RTE	Modelisation_Conso.py — coefficient_meteo()	84
Tous les prix EEX (3 jalons, Base et Peak)	scenarios.py — EEX	20
Prime de risque, volatilité CRE	scenarios.py	13-14
Le calcul de b	scenarios.py — pente_prix_volume()	159
Level × Shape	scenarios.py — scenario_spot()	172
Années de scénario	comparaison_scenarios.py — ANNEES	8
Prix de vente, marge cible, plancher	comparaison_scenarios.py	11-13
Mix par défaut (cal / trimestres / +peak)	comparaison_scenarios.py — FORME	15
Le scénario de choc	comparaison_scenarios.py	21-22
Les six stratégies	comparaison_scenarios.py — STRATEGIES	24
Fenêtres de livraison des produits	comparaison_scenarios.py — fenetre()	37
Pourcentages → MW par produit	comparaison_scenarios.py — construire_mix()	68
Le calcul de marge	comparaison_scenarios.py — marge()	160
Stratégie T1 retenue, options du T2	decision_t2.py	10-11
Température prévue, production solaire	spot_t3.py	16, 18
Ce qui est déjà acheté	spot_t3.py — POSITION	25
⚠ Production solaire annuelle	spot_t3.py — SOLAIRE_ANNUEL_MWH	21
⚠ Prix de clearing	spot_t3.py — PRIX_CLEARING	33

Les deux dernières lignes sont à `None` : ce sont les deux informations qui manquent pour terminer le T3.

Ce qui est actualisé à chaque jalon

Donnée	T1 (22/09)	T2 (01/12)	T3 (31/12)
Prix EEX	83,31	73,70	78,37
Effectifs clients	200 000 / 5 000	188 200 / 4 585	191 531 / 4 745
Volume 2027	536 627	501 962	512 820 MWh
Niveau des scénarios	ancré sur 83,31	ré-ancré sur 73,70	—
Probabilités	28/28/28/15 %	28/34/35/2 %	—
Position détenue	—	T1 figé	T1 + T2 figés

Ce qui ne bouge jamais, et c'est volontaire : les coefficients de profil Enedis et les gradients (ils décrivent un client, pas un marché), la température normale, les années historiques, la pente `b = +2,451`, et les quatre paramètres du Conseil.

Deux réserves à connaître. **b n'est jamais réestimé** : calculé une fois sur 454 jours d'hiver, il ne tient pas compte de l'hiver 2026. Défendable, puisqu'il décrit une relation structurelle et non un prix. Et **au T3 les scénarios ne sont pas ré-ancrés sur 78,37** : `spot_t3.py` raisonne sur le volume à température normale et les prix réels, ce qui en fait un

rééquilibrage de volume et non une ré-optimisation de marge. C'est la lecture littérale de l'objectif 1 (*evaluate what you should buy/sell*), pas une décision à la façon du §4.4.

Étapes 1 et 2 — Le profil Enedis et le FUD

Un profil Enedis est une **forme sans unité** ; le FUD est l'échelle qui la transforme en kilowatts.

La théorie

Enedis publie, pour chaque catégorie de client, une courbe de coefficients au pas demi-horaire qui dit comment un client moyen de cette catégorie répartit sa consommation dans l'année. Ces coefficients n'ont pas d'unité : ils décrivent une forme, pas un niveau.

Pour obtenir des kW il faut une échelle. Enedis la définit par le **Facteur d'Usage par Défaut** :

$$\mathrm{FUD} = P_s \times \Theta$$

où P_s est la puissance souscrite en kVA et Θ un coefficient publié par profil (document Enedis-NOI-NUM_09E). Θ est le taux d'utilisation moyen de la puissance souscrite.

Pour un client déjà en portefeuille, dont on connaît la consommation passée, on utiliserait le FUR (Facteur d'Usage Réel). Ici le portefeuille est fictif et sans historique, donc **FUD**. C'est le premier point à savoir justifier.

Les chiffres

Profil	Clients	Ps (kVA)	Θ	FUD (kW)	Puissance de référence
RES1	200 000	4,5	0,05210	0,23445	46 890 kW
PRO1	5 000	36,0	0,08004	2,88144	14 407 kW

Un client PRO1 pèse 12 fois un client RES1. Sur l'année, cela donne **76,7 % de l'énergie pour RES1 et 23,3 % pour PRO1**, parce que les deux profils n'ont pas la même forme (PRO1 consomme le jour, RES1 le soir).

Dans le code

```
PARAMS = {
    "RES1": {"nb_clients": 200_000, "puissance_souscrite": 4.5, "theta": 0.0521},
    "PRO1": {"nb_clients": 5_000, "puissance_souscrite": 36.0, "theta": 0.08004},
}

def fud(profil):
    p = PARAMS[profil]
    return p["puissance_souscrite"] * p["theta"]
```

Le 1er décembre, le service commercial revoit sa prévision à la baisse : **188 200 clients RES1 au lieu de 200 000, et 4 585 PRO1 au lieu de 5 000**.

Il n'y a rien à recalculer, parce que deux fois moins de clients consomment exactement deux fois moins, à chaque instant. Il suffit donc de multiplier la courbe par le **rapport des effectifs** : $188\,200 / 200\,000 = 0,941$ pour RES1, et $4\,585 / 5\,000 = 0,917$ pour PRO1. Ces deux facteurs s'appliquent à tous les pas de temps de l'année — à 3 h du matin le 14 juillet comme à 19 h le 7 janvier.

```
REVISION_T2 = {"RES1": -0.059, "PRO1": -0.083}

def reviser_t2(df):
    out = df.copy()
    for profil, variation in REVISION_T2.items():
        for col in (f"P_{profil}_kW", f"P_dyn_{profil}_kW"):
            out[col] = df[col] * (1 + variation)
    return _totaliser(out)
```

Ce qu'il faut savoir dire

- **Pourquoi FUD et pas FUR** : pas d'historique de consommation sur ce portefeuille.
- **D'où viennent les 0** : ce ne sont pas des valeurs inventées, elles sont lues dans le barème Enedis, profil par profil.
- **Pourquoi -5,96 % de clients donne -6,46 % d'énergie** : PRO1 ne pèse que 2,4 % des clients, mais 23,3 % de l'énergie. Comptée en nombre de clients, sa baisse de 8,3 % ne change presque rien ; comptée en énergie, elle compte dix fois plus lourd. D'où $0,976 \times (-5,9) + 0,024 \times (-8,3) = -5,96 \%$ en clients, contre $0,767 \times (-5,9) + 0,233 \times (-8,3) = -6,46 \%$ en énergie : ce sont les deux mêmes pourcentages, pondérés différemment. C'est exactement le genre de détail qu'on vous demandera.

Étape 3 — Charger et aligner les données

Quatre fichiers, deux séparateurs, deux résolutions : tout le travail consiste à les **joindre sur l'horodate** plutôt qu'à les coller côte à côte.

L'**horodate** — c'est le nom de la colonne chez Enedis, `HORODATE` — est la date et l'heure de début de chaque pas de temps, fuseau compris. `2025-01-07T18:30:00+01:00` désigne la demi-heure qui va de 18h30 à 19h00, en heure d'hiver française. C'est la seule chose que les quatre fichiers ont en commun, donc la clé qui permet de les rapprocher.

Ce qu'on charge

Fichier	Séparateur	Ce qu'on en tire
<code>RES1_BASE.csv</code> , <code>PRO1_BASE.csv</code>	virgule	<code>COEFFICIENT_PREPARE</code> (30 min) et <code>COEFFICIENT_DYNAMIQUE</code> (15 min)
<code>gradients.csv</code>	point-virgule	<code>RES1_BASE</code> , <code>PRO1_BASE</code> = gradient de thermosensibilité, en %/°C
<code>temperature_2025.csv</code>	virgule	température réalisée et température normale, lissées

Un coefficient de profil répond à une seule question : **à cet instant, un client moyen de cette classe consomme combien de fois sa consommation moyenne de l'année ?** C'est un multiplicateur sans unité — `1` veut dire « exactement sa moyenne », `2` « le double », `0,5` « la moitié ». Enedis mesure un échantillon de clients de la classe et normalise la courbe pour que sa moyenne annuelle vaille 1 : le coefficient ne porte donc **que la forme**, jamais le niveau. Celui de `RES1` va de 0,49 un matin de juin à 2,01 le 29 décembre à 19h30, soit un rapport de 4 entre le creux et la pointe.

Le fichier Enedis contient **trois** colonnes de coefficients, pas une :

Colonne	Résolution réelle	Moyenne 2025	Disponible
<code>COEFFICIENT_PREPARE</code>	30 min	1,0021	à l'avance
<code>COEFFICIENT_AJUSTE</code>	30 min	0,9959	après coup
<code>COEFFICIENT_DYNAMIQUE</code>	15 min	0,8772	après coup

On prend le **préparé** pour deux raisons. D'abord, c'est le seul qui existe *avant* la période : les deux autres sont des corrections établies une fois les relevés consolidés, donc inutilisables pour prévoir 2027. Ensuite, c'est celui qui est normalisé autour de 1, ce qui fait de `N × FUD` la puissance moyenne du portefeuille. Le dynamique, lui, tourne autour de 0,877 : l'utiliser tel quel donnerait 360 299 MWh au lieu de 411 616, soit **12,5 % du volume envolé**.

Reste un problème à régler : le préparé est au pas de 30 minutes, le dynamique au quart d'heure.

Affiner sans changer le niveau

Le coefficient préparé donne **une valeur par demi-heure**, le dynamique **deux valeurs par demi-heure** (une par quart d'heure). On veut la finesse du second sans toucher au niveau du premier : on lui emprunte donc uniquement **sa façon de répartir la demi-heure entre ses deux quarts d'heure**, et on applique cette répartition au préparé.

```
def affiner_au_quart_heure(demi_horaire, quart_horaire):
    demi_heure = quart_horaire.index.tz_convert("UTC").floor("30min")
    ratio = quart_horaire / quart_horaire.groupby(demi_heure).transform("mean")
    return demi_horaire * ratio
```

Le ratio vaut **1 en moyenne sur chaque demi-heure**, donc l'énergie est conservée exactement : on ne fait que la redistribuer entre les deux quarts d'heure. C'est la seule façon honnête de passer de 30 min à 15 min.

Le fuseau horaire

```
def _horodate(serie):  
    return pd.to_datetime(serie, format="ISO8601", utc=True).dt.tz_convert("Europe/Paris")
```

Chaque horodate **garde son fuseau** au lieu d'être une simple date-heure. C'est indispensable à cause des changements d'heure : dans la nuit d'octobre, **2h30 arrive deux fois**, et une date-heure sans fuseau ne permet pas de distinguer la première de la seconde ; dans la nuit de mars, 2h30 n'existe pas du tout. Sans fuseau, on se retrouve avec 35 096 lignes au lieu de 35 040 et des doublons invisibles à l'œil. C'est aussi pourquoi le `.floor("30min")` ci-dessus travaille en UTC : pandas refuse d'arrondir une heure ambiguë, et en UTC l'ambiguïté n'existe pas.

Au passage : $35\,040 = 365 \times 96$, soit 96 pas de 15 minutes par jour.

La jointure, et pourquoi c'est le point crucial

```
def construire_base():  
    df = charger_coefficients("RES1")  
    for autre in (charger_coefficients("PRO1"), charger_gradients(), charger_temperatures()):  
        df = df.merge(autre, on="HORODATE", how="inner")  
    return df.set_index("HORODATE").sort_index()
```

`merge(on="HORODATE")` veut dire : **aligne les lignes qui portent la même horodate**. Une concaténation par position dirait au contraire « première ligne avec première ligne, deuxième avec deuxième » — et c'est exactement ce que fait un copier-coller dans Excel.

Sur ces données, la différence est fatale. Ouvrez `temperature_2025.csv` : sa première ligne de données est le **31 décembre à 23h45**, sa dernière le **1er janvier à 00h00**. Le fichier est trié à l'envers. Collé par position à côté des coefficients Enedis — qui, eux, commencent au 1er janvier — il met les températures de fin décembre en face du début janvier, et ainsi de suite sur toute l'année.

Le piège, c'est que **le résultat reste plausible** : on obtient des courbes, des saisons, des hivers froids. Rien ne saute aux yeux. C'est l'erreur du classeur Excel du prof, et une jointure sur l'horodate y est immunisée par construction : si une ligne n'a pas de correspondante, elle disparaît au lieu de se tromper de voisine.

Les fichiers de prix historiques posent un autre problème : **ENTSO-E a changé son format d'export entre 2021 et 2024**. Les deux premières lignes de chaque fichier suffisent à le voir.

```
2021 : "MTU (CET/CEST)", "Day-ahead Price [EUR/MWh]", "Currency", "BZN|FR"  
      "01.01.2021 00:00 - 01.01.2021 01:00", "50.87", "EUR"  
  
2024 : "MTU (UTC)", "Area", "Sequence", "Day-ahead Price (EUR/MWh)", "..."  
      "01/01/2024 00:00:00 - 01/01/2024 01:00:00", "BZN|FR", "..."
```

Trois différences, toutes piégeuses :

- **Les guillemets.** En 2024, la ligne entière est entourée de guillemets et ceux de l'intérieur sont doublés. Résultat : pandas lit tout le fichier comme **une seule colonne**. Le code détecte ce cas et relit le fichier une deuxième fois après avoir défilé cet emballage.
- **La date.** `01.01.2021 00:00` (points, sans secondes) contre `01/01/2024 00:00:00` (barres, avec secondes). Le code essaie les formats connus l'un après l'autre et garde le premier qui convertit toutes les lignes sans échec.
- **Le fuseau.** `MTU (CET/CEST)` signifie heure de Paris, `MTU (UTC)` heure universelle — deux heures d'écart en été. Le code lit le fuseau **dans le titre de la colonne**, parce que c'est la seule indication fiable du fichier.

Rien de tout cela n'est intellectuellement passionnant, mais une erreur de fuseau décale une année entière de prix d'une heure : la pointe du soir se retrouve en face de la consommation de l'après-midi, et la régression prix-volume de l'étape 8 est faussée sans que rien ne casse.

Ce qu'il faut savoir dire

- Pourquoi le coefficient **préparé** est le bon pour de la prévision.
- Pourquoi l'affinage au quart d'heure **conserve l'énergie** (le ratio moyen vaut 1).
- Pourquoi on joint sur l'horodate : donner l'exemple des températures inversées, c'est l'argument le plus parlant du dossier.

Étapes 4 et 5 — Le coefficient météo et la charge

Le coefficient météo corrige le profil de l'écart entre température réalisée et température normale, **mais seulement sous le seuil de chauffe de 15 °C**.

La théorie

Une partie du parc français se chauffe à l'électricité : quand il fait plus froid que la normale, la consommation monte. RTE quantifie cette **thermosensibilité** par un gradient en %/°C, qui dépend du profil, de l'heure et de la saison. Fait important : **les gradients sont nuls d'avril à octobre** — pas de chauffage, pas de sensibilité.

g est ce gradient. Il est lu dans `gradients.csv` (colonnes `RES1_BASE` et `PRO1_BASE`) et porte le nom `grad_RES1 / grad_PRO1` dans le code. Attention à son unité : il est exprimé **en fraction par degré**, pas en pourcentage. `0,0172` veut dire 1,72 % de consommation en plus par degré perdu — c'est ce qui permet d'écrire $1 + g \times \Delta T$ directement, sans diviser par 100.

Profil	Hiver (jan-mars, nov-déc)	Avril à octobre	Maximum de l'année
RES1	2,2 à 2,5 %/°C	0	3,83 %/°C le 31/03 à 18h45
PRO1	3,0 à 3,7 %/°C	0	5,43 %/°C le 01/11 à 11h30

Trois remarques. `g` n'est pas une constante : il y a une valeur par pas de 15 minutes, comme pour les coefficients de profil. Il vaut **exactement zéro d'avril à octobre**, ce qui force `CM = 1` sur toute cette période quelle que soit la température. Et PRO1 est plus thermosensible que RES1 à puissance égale — un local professionnel chauffe un plus grand volume par rapport à sa consommation de base.

Au-dessus de 15 °C personne ne chauffe. On ne compte donc que la part de l'écart de température située **sous le seuil**, ce qui donne quatre cas :

Cas	T réalisée	T normale	CM	Lecture
1	< 15 °C	< 15 °C	$1 + g \times (T_n - T)$	hiver normal : tout l'écart compte
2	< 15 °C	≥ 15 °C	$1 + g \times (T_s - T)$	froid inattendu : seule la part sous 15 compte
3	≥ 15 °C	< 15 °C	$1 + g \times (T_n - T_s)$	doux inattendu : on retire la chauffe prévue
4	≥ 15 °C	≥ 15 °C	1	été : aucune correction

Deux piquets à planter : le seuil existe (sans lui, on applique un gradient en plein mois d'août), et **CM est multiplicatif, pas additif**. L'oublier gonflait notre pointe de 14,5 % et notre énergie de 13,6 %.

```
def coefficient_meteo(gradient, t_normale, t_realisee, t_seuil=T_SEUIL):
    T, Tn, Ts = t_realisee, t_normale, t_seuil
    cm = np.select(
        [(T < Ts) & (Tn < Ts), (T < Ts) & (Tn >= Ts), (Tn < Ts) & (T >= Ts), (T >= Ts) & (Tn >= Ts)],
        [1 + gradient * (Tn - T), 1 + gradient * (Ts - T), 1 + gradient * (Tn - Ts), 1.0],
        default=np.nan)
    return pd.Series(cm, index=t_realisee.index)
```

`np.select` est un SI imbriqué appliqué d'un coup aux 35 040 lignes : c'est la version vectorisée du tableau ci-dessus, dans le même ordre.

La charge

$$P(t) = N_{\{\text{clients}\}} \times \mathrm{FUD} \times c_{\{\text{profil}\}}(t) \times \mathrm{CM}(t)$$

```

for profil in ("RES1", "PRO1"):
    f, nb = fud(profil), PARAMS[profil]["nb_clients"]
    coef = out[f"coef_{profil}"]
    out[f"CM_{profil}"] = coefficient_meteo(
        out[f"grad_{profil}"], out["temperature_normale_lissee_degc"],
        out["temperature_realisee_lissee_degc"])
    out[f"P_{profil}_kW"] = nb * f * coef
    out[f"P_dyn_{profil}_kW"] = nb * f * coef * out[f"CM_{profil}"]

```

Le code produit **deux colonnes par profil**, et c'est tout le but :

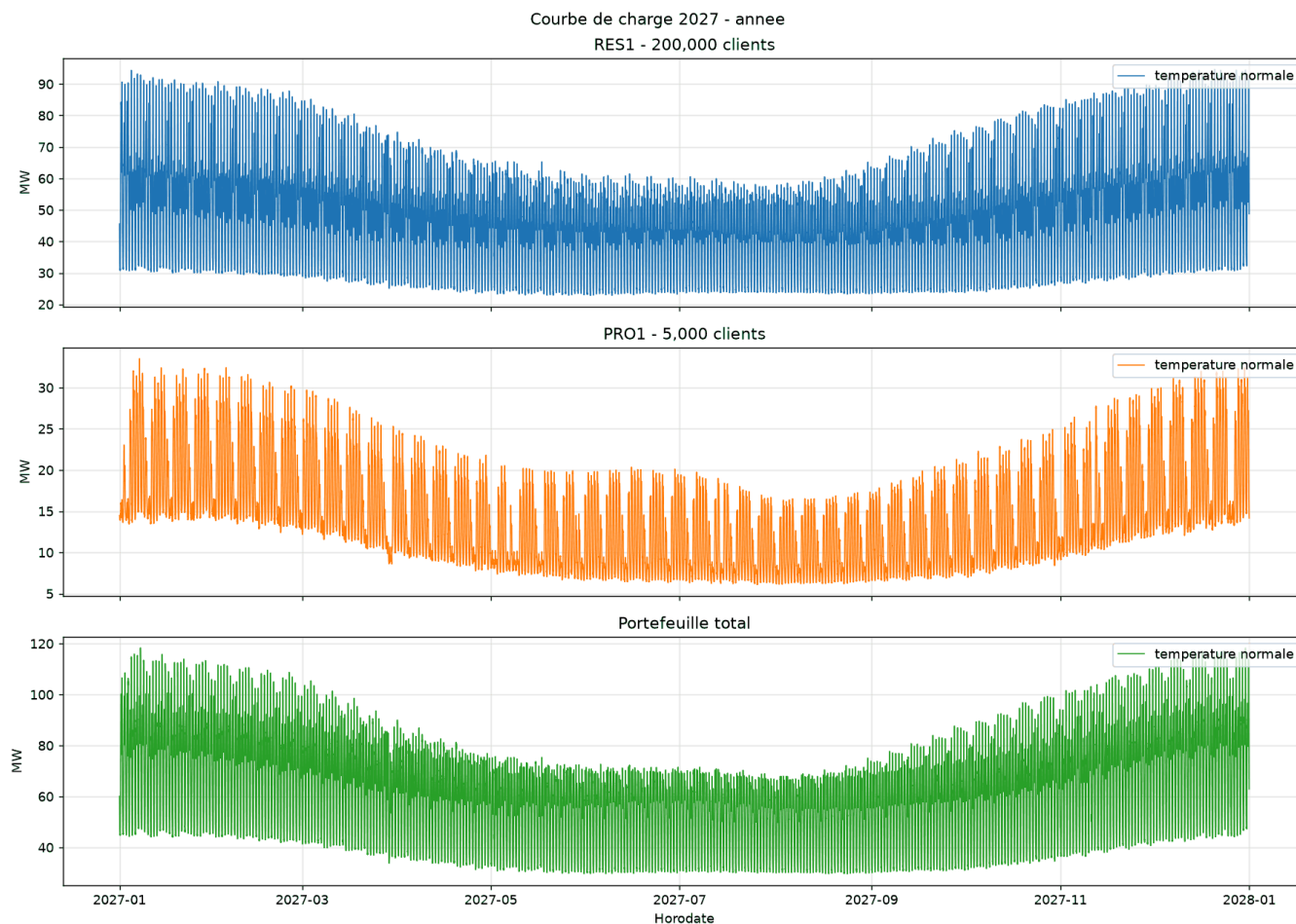
- `P_*_kW` — à température normale. C'est le **volume de référence, le 100 % contre lequel tous les ratios de couverture sont exprimés**, comme le §3.1 l'exige.
- `P_dyn_*_kW` — avec la météo du scénario. C'est ce qu'on consomme vraiment.

Ce qui sort, et pourquoi c'est plausible

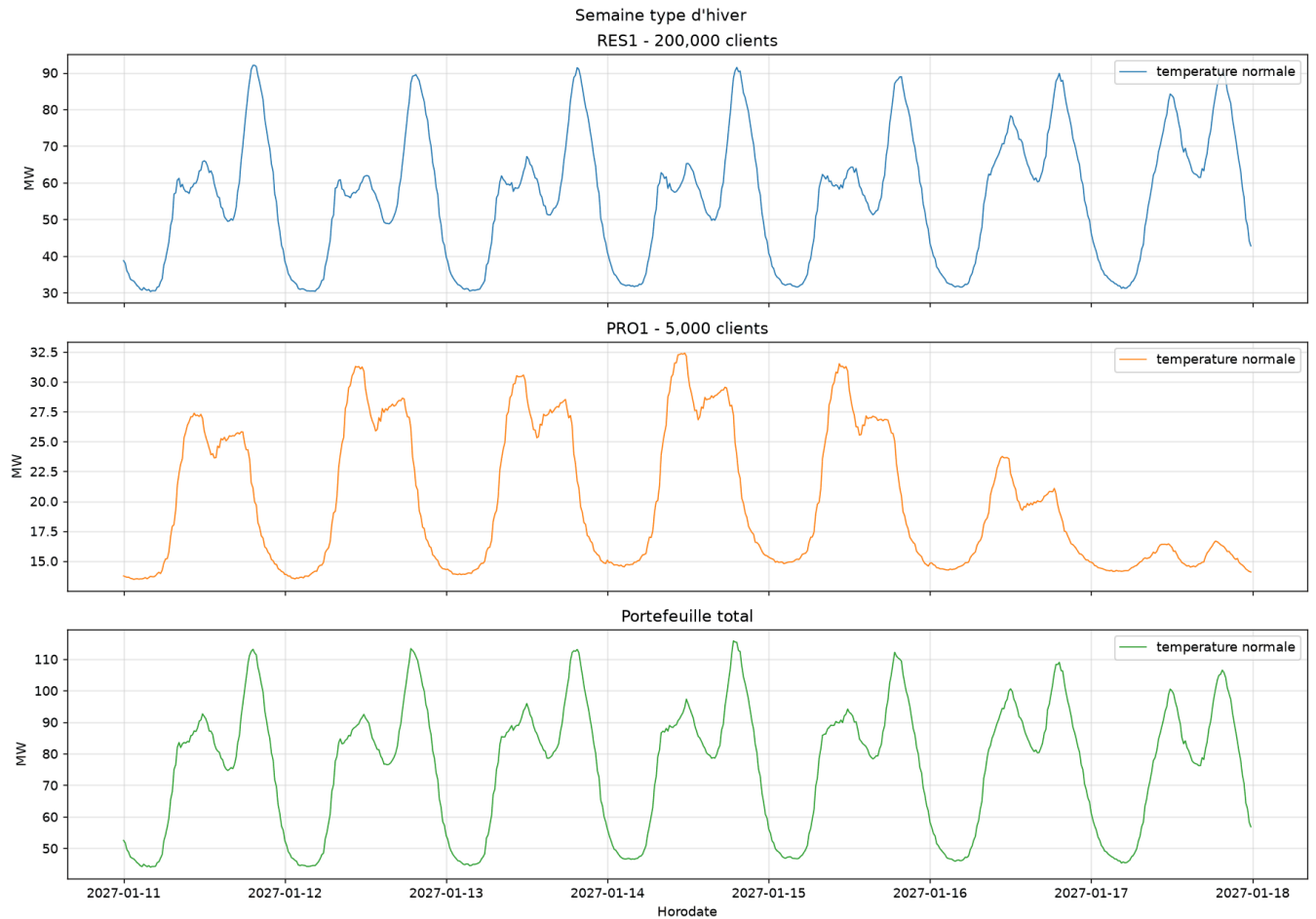
Résultat 2027, température normale	Valeur
Énergie annuelle	536 627 MWh
Pointe	118,33 MW le 07/01 à 18h45
Peak / off-peak	42,4 % / 57,6 %
Thermosensibilité moyenne de chauffe	1,96 MW/°C
Thermosensibilité maximale	3,73 MW/°C le 05/11 à 18h45

Les figures que trace le fichier

`Modelisation_Conso.py` trace trois figures quand on le lance. Elles ne servent pas à décorer : elles servent à vérifier d'un coup d'œil que la charge a la bonne tête, avant de calculer quoi que ce soit dessus.



L'année entière, 17 520 points par profil. Le trait paraît épais parce que chaque journée monte et redescend d'environ 40 MW : à cette échelle, l'œil ne voit que l'enveloppe. Trois choses s'y lisent. **La saisonnalité** : le total culmine au-dessus de 115 MW début janvier et retombe dans une bande de 35 à 70 MW en août — c'est la thermosensibilité de l'étape 4, et comme aucun des deux profils n'a de climatisation, l'été est un plancher, pas un deuxième pic. **Le poids relatif des deux profils** : RES1 tient l'essentiel (20 à 93 MW), PRO1 complète (5 à 33 MW). **Le resserrement d'avril à octobre** : c'est la période où $g = 0$ et où CM vaut exactement 1 ; toute la variabilité qui reste vient du calendrier, plus de la météo.



Une semaine d'hiver (lundi 11 au dimanche 17 janvier 2027) : la même courbe, dézoomée. C'est là que la forme Enedis devient lisible, et que la différence entre les deux profils saute aux yeux. RES1 fait deux bosses par jour — une le matin, une plus haute le soir qui monte à 90 MW — et retombe à 31 MW entre 3h et 6h. PRO1 fait un plateau de bureau : montée brutale vers 7h, 25 à 30 MW toute la journée, retour à 14 MW le soir. Le week-end, les deux divergent franchement : samedi et dimanche, PRO1 plafonne à 24 puis 17 MW, pendant que RES1 garde des pointes de 80 à 90 MW.

Ce décalage est exactement ce que mesure le ratio peak / off-peak du paragraphe suivant.

Peak et Off-peak ne sont pas les heures pleines et creuses d'une facture de client : ce sont les deux fenêtres de livraison des produits échangés sur **EEX** — l'*European Energy Exchange*, la bourse européenne de l'électricité basée à Leipzig, où se négocient les contrats à terme avec lesquels on se couvre. C'est de son site que viennent les prix repris dans `scenarios.py`.

	Fenêtre	Heures/an	Part des heures	Part de notre énergie
Peak	lundi-vendredi 08h-20h, jours fériés inclus	3 132 h	35,8 %	42,4 %
Off-peak	tout le reste, week-ends compris	5 628 h	64,2 %	57,6 %

Les heures Peak portent donc **plus d'énergie que leur part de temps** : 72,67 MW de puissance moyenne contre 54,91 MW en Off-peak, soit un **ratio de forme de 1,32**. Un portefeuille parfaitement plat afficherait 1,00. C'est ce 1,32 qui servira à l'étape 11 à dimensionner un éventuel bloc Peak, et le §3.1 réclame nommément ce partage.

À ne pas confondre, donc : les heures creuses d'Enedis durent 8 h par jour, s'appliquent aussi le week-end et varient d'une commune à l'autre. Elles servent à facturer un client, pas à acheter sur un marché. Nos clients sont d'ailleurs au tarif base — la colonne `SOUS_PROFIL` du fichier Enedis indique `RES1_BASE` et `PRO1_BASE`.

Deux contrôles de bon sens qui valent de l'or en soutenance :

- **Le foisonnement.** La somme des pointes individuelles des deux profils vaut 136,98 MW, celle du portefeuille 125,40 MW : **8,5 % d'écart**, parce que RES1 pointe à 19h30 et PRO1 à 10h30. Un portefeuille n'est jamais la somme de ses pointes.
- **L'ordre de grandeur de la thermosensibilité.** RTE annonce environ 2 400 MW/°C pour la France entière. Notre portefeuille pèse de l'ordre du millième de la consommation nationale, et on trouve 1,96 MW/°C. C'est cohérent, et c'est exactement le « checking that the curve behaves sensibly » qui fait passer l'étape 1 en Outstanding.

Étapes 6 et 7 — Projeter sur 2027, rejouer une météo

Deux alignements coexistent dans le même tableau, et c'est volontaire : **la consommation se recale par type de jour, la température par date calendaire.**

Pourquoi deux appariements

Les données Enedis portent sur 2025, le livrable sur 2027. Il faut transporter la forme d'une année à l'autre, mais les deux grandeurs ne voyagent pas de la même façon.

- **Un profil de consommation suit le type de jour.** Un mardi se comporte comme un mardi. Le 1er janvier 2025 était un mercredi, le 1er janvier 2027 est un vendredi : recaler par date collerait un profil de mercredi sur un vendredi.
- **La climatologie suit la date.** Le 15 janvier a la même normale saisonnière tous les ans, quel que soit le jour de la semaine.

Le §3.1 l'exige explicitement : *align historical years on the 2027 calendar by weekday, not by date, and move public holidays to their 2027 dates.*

Sept types de jour

```
def _type_jour(d, ferries):
    return 7 if d in ferries else d.weekday()
```

Lundi (0) à dimanche (6), **plus une 7^e catégorie pour les fériés**. Un férié n'est ni un mardi ni un dimanche : les gens sont chez eux, mais les commerces ne sont pas fermés comme un dimanche. Lui donner sa propre classe, c'est ce qui permettra, à l'étape 14, de prévoir le 1er janvier 2027 avec un profil de férié et pas de vendredi — ce que le §5.2 demande nommément.

```
def apparier_jours(an_source, an_cible):
    for d in cib:
        candidats = par_type.get(_type_jour(d, f_cib)) or par_type[6]
        ref = d.timetuple().tm_yday
        appariement[d] = min(candidats, key=lambda s: min(
            abs(s.timetuple().tm_yday - ref), 365 - abs(s.timetuple().tm_yday - ref)))
```

Pour chaque jour de 2027 on prend le jour de 2025 **de même type le plus proche dans l'année**. La distance est circulaire (`365 - abs(...)`) pour que le 31 décembre soit voisin du 1er janvier. Le repli `or par_type[6]` couvre le cas où un type n'existerait pas dans l'année source : on retombe sur un dimanche. Résultat : 365 jours sur 365 appariés au bon type.

Les fériés mobiles

Pâques bouge, donc trois fériés sur onze bougent avec elle. Le code calcule la date par l'algorithme de Meeus/Jones/Butcher :

```
def _paques(annee):
    a = annee % 19
    b, c = divmod(annee, 100)
    ...
    mois, jour = divmod(h + l - 7 * m + 114, 31)
    return dt.date(annee, mois, jour + 1)
```

Validé sur huit dates connues (2000, 2024 à 2028, 2030, 2038). De là se déduisent le lundi de Pâques (+1 jour), l'Ascension (+39) et le lundi de Pentecôte (+50), auxquels s'ajoutent les huit fériés fixes.

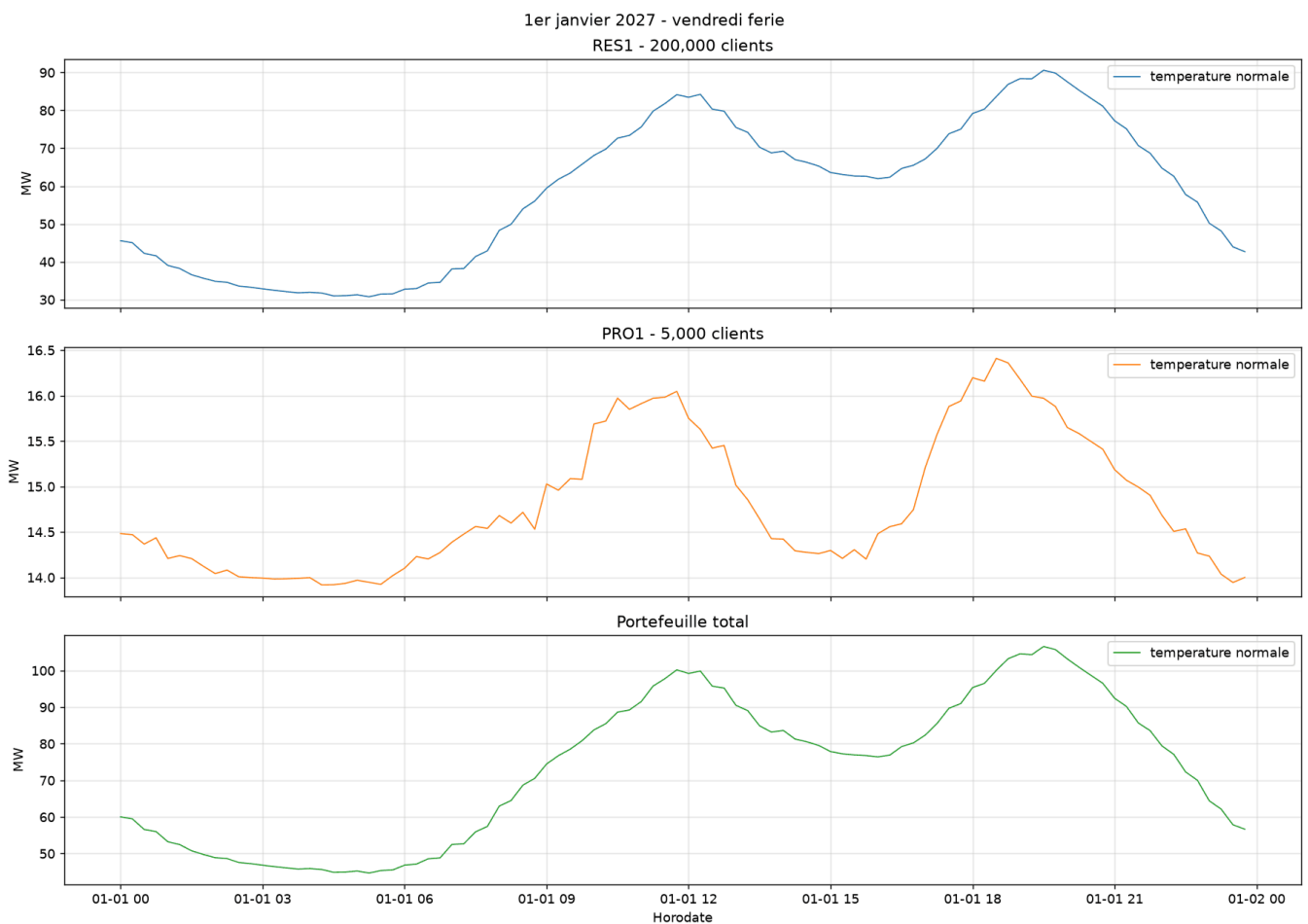
Deux piquets techniques

L'année bissextile. Projeter sur une année bissextile depuis une année qui ne l'est pas fait planter la construction du 29 février :

```
try:
    jumeau = dt.date(an_source, d.month, d.day)
except ValueError:          # 29 fevrier cible, source non bissextile
    jumeau = dt.date(an_source, d.month, d.day - 1)
```

Les changements d'heure. `projeter()` reconstruit un index 2027 **neuf**, donc avec les bons changements d'heure de 2027, puis assemble jour par jour en recopiant la position dans la journée. Un jour de 23 h ne peut pas remplir un jour de 25 h, d'où le prolongement :

```
if len(src) < n:
    src = pd.concat([src] + [src.iloc[[-1]]] * (n - len(src)))
bloc = src.iloc[:n].copy()
bloc.index = pd.DatetimeIndex(lignes)
```



La troisième figure du fichier est la preuve que l'appariement par type de jour fonctionne. Le 1er janvier 2027 tombe un vendredi. Apparié par date calendaire, il aurait hérité d'un vendredi ouvré de 2025 et PRO1 serait monté à 30-32 MW dans la journée, comme sur la semaine type plus haut. Ici PRO1 ne dépasse jamais **16,4 MW** : `_type_jour()` lui a donné le code 7, `appariier_jours()` est allé chercher un autre jour férié de 2025, et le profil obtenu est bien celui d'un jour chômé.

RES1 raconte la même chose autrement : pas de creux marqué en milieu de journée, deux bosses larges (84 MW vers 12h, 90 MW vers 19h) au lieu du double pic serré d'un jour ouvré. Le portefeuille total culmine à **106 MW à 19h** et descend à 45 MW vers 5h.

L'erreur évitée se chiffre : une quinzaine de MW sur les heures de bureau, soit de l'ordre de 150 MWh sur la seule journée. Et comme c'est précisément cette journée que l'étape 14 utilise pour construire la position ouverte, l'erreur se serait propagée jusqu'à l'ordre day-ahead.

Étape 7 — injecter une météo passée

Même mécanique, autre grandeur : on prend la température **réalisée** d'une année passée (2021, 2023, 2024) et on la pose sur le calendrier 2027, par date.

```
def injecter_temperature(cible, source):
    out = cible.copy()
    valeurs = _reindexer_par_calendrier(source, out.index)
    valeurs = valeurs.fillna(out["temperature_normale_lissee_degc"])
    out["temperature_realisee_lissee_degc"] = valeurs.to_numpy()
    return out
```

La réindexation passe par une clé entière et un `searchsorted` plutôt que par un index à plusieurs niveaux, précisément pour que les heures doublées du changement d'heure ne cassent rien :

```
def _cle_calendaire(index):
    return (index.month * 1_000_000 + index.day * 10_000
            + index.hour * 60 + index.minute)
```

À partir d'ici, la température réalisée n'est plus égale à la normale : `CM` s'écarte de 1, et `P_dyn*_kW` se sépare de `P*_kW`. Le scénario commence.

Ce qu'il faut savoir dire

- Pourquoi **deux appariements différents** dans le même module — c'est la question qu'on vous posera.
- Pourquoi un férié mérite sa propre catégorie, et ce que ça rapporte au §5.
- Que l'index 2027 est **reconstruit**, pas décalé : c'est ce qui garantit 35 040 pas et pas 35 096.

Étapes 8 et 9 — De l'anomalie de volume à la courbe de prix

Quand il fait froid, notre portefeuille consomme plus **et** le spot monte : on achète davantage, plus cher. Toute cette partie sert à mettre un chiffre sur ce cumul.

Étape 8 — la pente b

Le risque n'est pas le volume seul ni le prix seul, c'est leur **corrélation défavorable**. On la mesure en trois temps :

1. **Anomalie de volume** du jour = $(\text{volume réel} - \text{volume normal}) / \text{volume normal}$.
2. **Anomalie de prix** du jour = $(\text{prix du jour} - \text{moyenne du mois}) / \text{moyenne du mois}$. On retire la moyenne mensuelle pour éliminer la tendance de niveau et ne garder que l'écart propre au jour.
3. **Régresser** la seconde sur la première, **sur les mois d'hiver uniquement** (janvier, février, mars, novembre, décembre) : ailleurs le gradient est nul, donc l'anomalie de volume l'est aussi et la régression n'aurait aucun sens.

Un exemple pour fixer les idées (scénario 2021). Le 13 février, le portefeuille consomme 1 974,96 MWh là où la température normale en prévoyait 1 728,62 : anomalie de volume de **+14,3 %**.

Côté prix, la normalisation mensuelle n'est pas un raffinement. La moyenne spot passe de 49,01 €/MWh en février à 274,67 € en décembre 2021 — un facteur 5,6 qui est celui de la crise gazière, pas de la météo.

Jour	Prix brut	Moyenne de son mois	Anomalie
11 février	79,01 €/MWh	49,01	+61 %
30 décembre	105,74 €/MWh	274,67	−61 %

Le 30 décembre coûte **plus cher en valeur absolue** et reste pourtant un jour bon marché. Sans retirer la moyenne de son mois, on conclurait exactement l'inverse.

Quant à la restriction à l'hiver, ce n'est pas une précaution mais une nécessité arithmétique : d'avril à octobre $g = 0$, donc le volume réel égale **exactement** le volume normal et l'anomalie vaut 0,000 tous les jours. `np.var(x)` serait nul, et $b = \text{Cov} / \text{Var}$ **diviserait par zéro**.

Enfin, le lien est loin d'être parfait, et il faut savoir le dire : les 1er, 2 et 3 janvier 2021 affichent un volume en hausse (+3 à +5 %) et un prix en baisse (−15 à −21 %). Ce sont les fériés du Nouvel An — l'industrie est à l'arrêt et la demande nationale s'effondre, pendant que nos clients résidentiels chauffent toujours.

```
prix_jour = prix_horaire.groupby(prix_horaire.index.date).mean()
moyenne_mois = prix_jour.groupby(prix_jour.index.month).transform("mean")
tab["anomalie_prix"] = (prix_jour - moyenne_mois) / moyenne_mois
tab["anomalie_volume"] = (tab["volume_reel"] - tab["volume_normal"]) / tab["volume_normal"]
```

```
def pente_prix_volume(tab):
    x = tab["anomalie_volume"].to_numpy()
    y = tab["anomalie_prix"].to_numpy()
    b = np.cov(x, y, ddof=0)[0, 1] / np.var(x)
    r = np.corrcoef(x, y)[0, 1]
    residus = y - (y.mean() + b * (x - x.mean()))
    se = np.sqrt((residus ** 2).sum() / (n - 2) / ((x - x.mean()) ** 2).sum())
    return {"b": b, "r": r, "r2": r ** 2, "n": n, "se": se, "t": b / se}
```

$b = \text{Cov}(x, y) / \text{Var}(x)$ est la pente des moindres carrés, écrite directement. Les trois sorties suivantes répondent à l'exigence du §3.2 (*check the quality of the regression*).

Scénario	b	r	r ²	t	n jours
2021	+1,930	+0,499	0,249	+7,0	151
2023	+3,620	+0,623	0,388	+9,7	151
2024	+2,023	+0,371	0,138	+4,9	152
Mutualisée	+2,451	+0,493	0,243	+12,1	454

Comment lire b = +2,451 : un jour où notre volume est 1 % au-dessus de la normale, le spot est en moyenne 2,45 % au-dessus de la moyenne de son mois.

Comment lire le reste : le t de +12,1 dit que la pente n'est pas un hasard. Mais le r² de 0,243 dit que **le volume n'explique qu'un quart de la variance du prix** — les trois autres quarts viennent du gaz, du nucléaire, du vent. C'est précisément pour ça que le code teste ensuite la sensibilité de la décision à b, et c'est l'honnêteté qui est valorisée ici.

Pourquoi mutualiser : les b annuels vont de 1,93 à 3,62, presque du simple au double. Une pente estimée sur 454 jours est plus stable que n'importe laquelle des trois.

Étape 9 — Level × Shape

La règle du cours : **on ne réutilise jamais le niveau de prix historique**. 2021 moyennait 109 €/MWh, 2024 seulement 58 €/MWh ; ces niveaux appartiennent à leur année et ne disent rien de 2027. Ce qu'on garde d'une année passée, c'est sa **forme** : la façon dont le prix se répartit entre les heures et les jours.

Trois mots à poser avant d'aller plus loin.

Un **forward** est un contrat qui fixe aujourd'hui le prix d'une livraison future. Acheter le Cal-27 Base à 83,31 €/MWh le 22 septembre 2026, c'est s'engager à payer ce prix pour de l'électricité livrée tout au long de 2027, quel que soit le prix **spot** — le prix du marché au jour le jour — une fois l'échéance venue. C'est l'instrument même de la couverture : il supprime l'incertitude sur le prix, en échange de quoi on renonce à profiter d'une baisse.

La **prime de risque** π est l'écart entre ce prix et ce que le marché anticipe réellement. Celui qui vous vend un forward porte le risque à votre place, et il se fait payer pour ça : le forward se situe donc systématiquement **un peu au-dessus de l'espérance du spot**. Le cours fixe cette prime à 3 %, et le code la nomme `PRIME_DE_RISQUE`.

Le **Level** est le prix spot **moyen** retenu pour toute l'année 2027 dans un scénario donné. Un seul nombre par scénario — 80,72 pour 2021, 78,00 pour 2023, 77,46 pour 2024 — autour duquel la **Shape** fera ensuite osciller le prix heure par heure.

$$\text{Level} = F_{T1} \times (1 - \pi) \times (1 + b \times a)$$

Terme	Valeur	Ce qu'il apporte
<code>F_T1</code>	83,31 €/MWh	le niveau que le marché price au 22/09/2026
<code>(1 - pi)</code>	$\pi = 3 \%$	un forward n'est pas une prévision : c'est un prix payé pour que quelqu'un porte le risque. L'espérance du spot est en dessous.
<code>(1 + b x a)</code>	$b = 2,451$	corrige du lien prix-volume : un scénario froid a une anomalie <code>a</code> positive, donc un Level plus haut

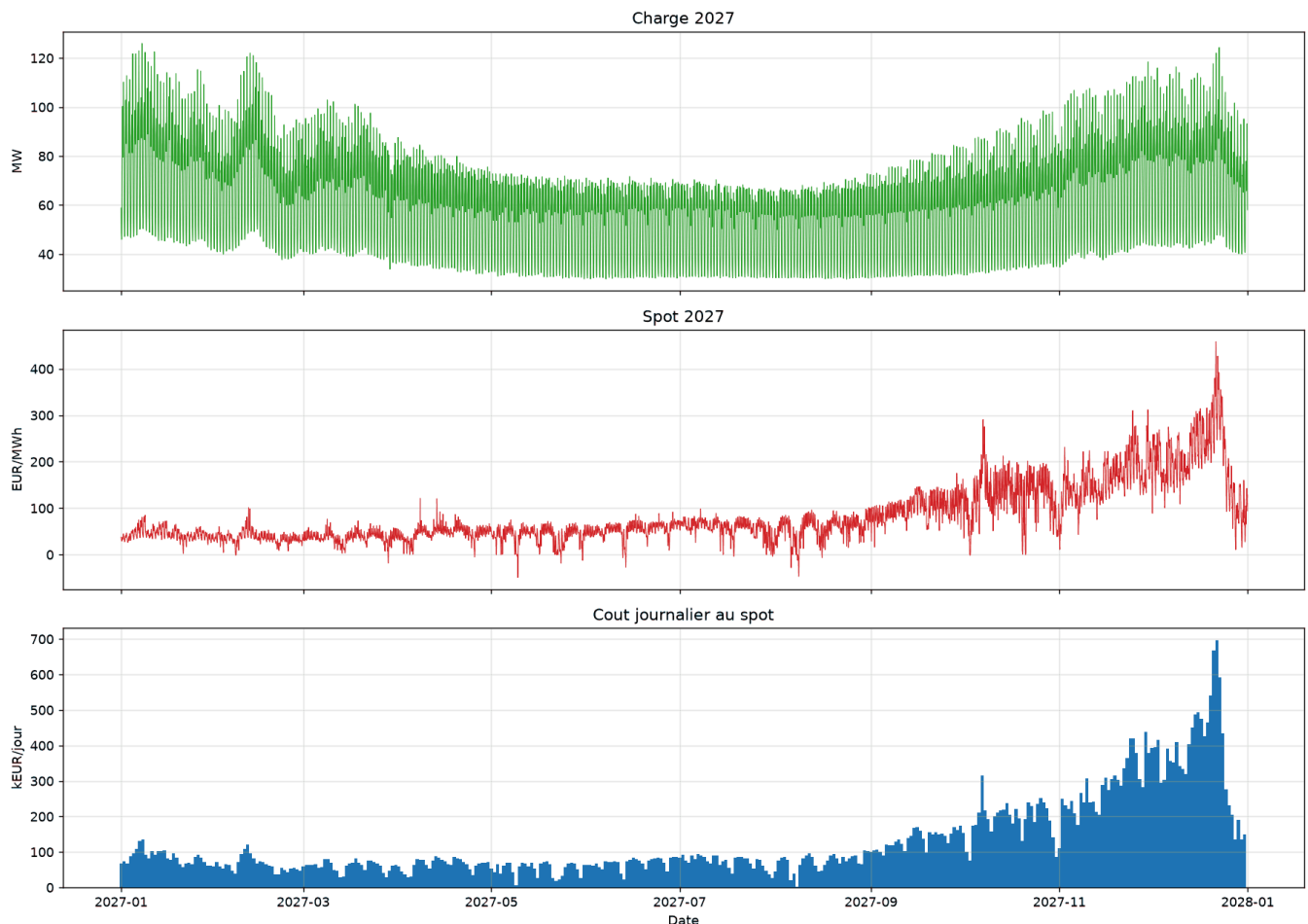
```
def scenario_spot(prix_hist, niveau, index_cible):
    forme = prix_hist / prix_hist.mean()
    valeurs = _reindexer_par_calendrier(forme, index_cible).fillna(1.0)
    return pd.Series(niveau * valeurs.to_numpy(), index=index_cible, name="spot")
```

`forme` a une moyenne de 1 **par construction**. Multipliée par le Level, elle donne une courbe 2027 dont la moyenne vaut exactement le Level. C'est tout le mécanisme.

Scénario	T moyenne	Anomalie volume	Level	Prix pondéré	Surcoût
2021	12,68 °C	−0,05 %	80,72	87,37	+6,65
2023	14,17 °C	−1,42 %	78,00	82,17	+4,18
2024	12,92 °C	−1,69 %	77,46	83,42	+5,98

Le chiffre à retenir

Le **prix pondéré** est le vrai coût moyen d'approvisionnement : $\Sigma(\text{charge} \times \text{prix}) / \Sigma \text{charge}$. Il est systématiquement au-dessus du prix moyen simple, parce qu'on consomme davantage aux heures où le prix est haut. L'écart entre les deux, **+4,18 à +6,65 €/MWh selon le scénario, est le coût du risque prix-volume**. C'est le chiffre qui justifie à lui seul qu'on se couvre.



C'est la figure que trace `scenarios.py`, ici sur le scénario 2021 (`ANNEE_HISTORIQUE = 2021`). Les trois panneaux partagent le même axe de temps, et c'est tout l'intérêt : on lit la charge, le prix, puis le produit des deux, l'un sous l'autre.

Le panneau du milieu, c'est la crise de 2021 telle qu'elle a eu lieu : un spot entre 20 et 100 €/MWh jusqu'en août, quelques heures négatives au printemps, puis une montée continue jusqu'à **460 €/MWh fin décembre**. Le panneau du bas est le coût journalier au spot, et il ne ressemble ni à la charge ni au prix : il ressemble à leur produit. Les journées à 700 k€ de décembre ne sont pas les plus chargées de l'année, ce sont celles où un prix extrême tombe sur une charge d'hiver.

C'est exactement l'écart entre prix moyen et prix pondéré décrit juste au-dessus, rendu visible : un coût total ne se déduit pas d'un prix moyen. C'est aussi pourquoi un seul scénario ne suffit pas — le même fichier lancé sur 2023 ou 2024 donne une figure très différente, d'où la comparaison de l'étape 12.

Ce qu'il faut savoir dire

- Pourquoi la forme est **normalisée à 1** et pourquoi on jette le niveau historique.
- Ce que vaut le r^2 et ce qu'il impose (tester la sensibilité à b , ne pas sur-vendre la régression).
- La différence entre prix moyen et prix pondéré : c'est la question piège classique.

Étape 10 — La méthode alpha

Au T1 on décide ce qu'on laisse pour le T2, sans connaître le prix du T2 : il faut donc en faire une **hypothèse assumée**, identique pour tous les scénarios.

La formule

$$F_{\{T2\}} = F_{\{T1\}} + \alpha \times (\text{Level} - F_{\{T1\}})$$

α est le **degré d'apprentissage du marché** entre le 22 septembre et le 1er décembre :

- $\alpha = 0$: au 1er décembre, le marché n'a rien appris sur 2027, le forward n'a pas bougé.
- $\alpha = 1$: au 1er décembre, le marché connaît déjà toute l'année 2027. Impossible.
- Entre les deux : une partie du scénario (perspectives d'hiver, stocks de gaz, premières prévisions de janvier) est déjà dans les prix.

```
def forward_t2_anticipe(niveau, alpha, forward_t1=FORWARD_CAL27_T1):
    return forward_t1 + alpha * (niveau - forward_t1)
```

La constante vit en tête de `comparaison_scenarios.py` et sert à tous les scénarios — le §3.3 exige qu'elle soit la même partout :

```
ALPHA_T2 = 0.50 # part du scenario deja dans les prix au 01/12/2026
```

Le contrôle de plausibilité

On vient de fabriquer un prix de T2 à partir d'une hypothèse. Rien ne garantit encore qu'il soit crédible, d'où la question à se poser : **le marché peut-il réellement bouger d'un tel montant en 70 jours ?**

La CRE donne de quoi répondre, avec une **volatilité annuelle de 20 %**. Comprendre : sur un an, le prix du Cal-27 s'écarte typiquement de 20 % de son point de départ, dans un sens **ou** dans l'autre. Ce n'est pas une prévision de direction, c'est une amplitude de vagabondage.

Sauf qu'il ne reste pas un an entre le T1 et le T2, mais 70 jours — **19 % d'une année**. Et un prix ne vagabonde pas proportionnellement au temps : il avance par pas aléatoires indépendants, si bien que c'est sa **variance** qui croît avec le temps, et donc son écart-type avec la **racine** du temps. Sur 70 jours, l'amplitude typique vaut $\sqrt{(70/365)} = 0,44$ fois celle d'une année entière.

```
def sigma_t2(forward_t1=FORWARD_CAL27_T1):
    return forward_t1 * VOLATILITE_CRE * np.sqrt(JOURS_T1_T2 / 365.0)
```

$83,31 \times 0,20 \times 0,438 = 7,30$ €/MWh. À un écart-type, le forward du 1er décembre avait donc toutes les chances de tomber dans **[76,01 ; 90,61]**. Tout ce qui sort largement de cette fourchette doit être justifié par autre chose que le hasard de marché.

Ce calcul n'est pas de notre invention : le prof le pose à l'identique en session 4 et cite ses deux sources — Hull, *Options, Futures and Other Derivatives*, pour la mise à l'échelle en racine du temps, et la délibération CRE n° 2026-06 pour les 20 %. Il annonce lui-même « environ ± 7 €/MWh sur 70 jours à 20 % de volatilité ».

Ce que ça a donné

Scénario	Level	$\alpha = 0,2$	$\alpha = 0,5$	$\alpha = 0,6$	α qu'il aurait fallu
2021	80,72	82,79	82,02	81,76	3,71
2023	78,00	82,25	80,66	80,12	1,81
2024	77,46	82,14	80,39	79,80	1,64

Prix réel observé au T2 : **73,70 €/MWh**.

La leçon, et c'est la plus intéressante du dossier

Aucune valeur de α entre 0 et 1 ne pouvait atteindre 73,70 : il aurait fallu 1,64 à 3,71. Ce n'est donc pas un mauvais choix de α , c'est un problème de nature : la baisse n'était pas d'origine météo.

Le contrôle CRE le confirme : 73,70 se situe à $-1,3$ sigma de 83,31, donc **parfaitement plausible comme mouvement de marché**, mais aucun scénario de température ne le décrit. C'est littéralement ce que le §3.3 attend d'un travail Outstanding : *naming the limit of the assumption — the forward moves on gas, CO₂ and nuclear availability, which no weather scenario describes*.

Autre façon de le dire, plus parlante : notre couloir d'anticipations allait de 80,39 à 82,02, soit **2 % de large**, face à une volatilité de 20 % annualisés. Il était beaucoup trop étroit. Dites-le avant qu'on vous le dise.

Ce qu'il faut savoir dire

- α est une **hypothèse**, pas une prévision, et elle doit être la même pour tous les scénarios.
- D'où vient le $\sqrt{t}$ dans sigma.
- Que l'erreur d'anticipation a une cause **nommable** (gaz, CO₂, nucléaire, demande), et qu'un modèle météo ne pouvait structurellement pas la voir.

Étape 11 — Le mix de produits EEX

« On couvre 70 % » ne veut rien dire pour un trader : ce qui s'achète, ce sont des **MW constants livrés sur une période définie**. Le §3.4 exige la conversion.

Les produits

Famille	Période de livraison	Heures
Cal-27	01/01 au 31/12/2027	8 760
Q1-27 à Q4-27	le trimestre	2 159 à 2 209
Jan-27 à Jun-27	le mois	variable

Chacun existe en **Base** (toutes les heures) et en **Peak** (lundi-vendredi 8h-20h, **jours fériés inclus** — retenez-le, ça ressort à l'étape 14).

La brique centrale

Une seule fonction dit, pour chaque pas de temps, si un produit livre ou non. Tout le reste en découle.

```
def fenetre(produit, nature, index):
    nom = produit.split("-")[0]
    if nom == "Cal":
        m = np.ones(len(index), dtype=bool)
    elif nom[0] == "Q":
        m = np.asarray(index.quarter) == int(nom[1:])
    else:
        m = np.asarray(index.month) == MOIS_EEX[nom]
    if nature == "Peak":          # lun-ven 08h-20h, jours ferries inclus
        m = m & np.asarray(masque_peak(index))
    return pd.Series(m, index=index)
```

```
def livraison(mix, index):
    out = pd.Series(0.0, index=index)
    for (produit, nature), mw in mix.items():
        out += mw * fenetre(produit, nature, index)
    return out

def cout_mix(mix, index, jalon="T1"):
    return sum(mw * sc.prix_eex(p, n, jalon) * heures_livrees(p, n, index)
               for (p, n), mw in mix.items())
```

Un mix est un simple dictionnaire `{(produit, nature): MW}`. `livraison()` donne la puissance couverte heure par heure, `cout_mix()` le coût fixe. **Ces deux fonctions servent jusqu'à la fin du modèle**, y compris pour le tableau 24 h du 1er janvier.

Passer d'une part à des MW

```
def construire_mix(part, volumes_q, index, forme=None, ratio_peak=1.32):
    if forme == "cal":
        return {"Cal-27", "Base": part * sum(volumes_q.values())
                / heures_livrees("Cal-27", "Base", index)}

    mix = {}
    for q, v in volumes_q.items():
        produit = f"Q{q}-27"
        h_base = heures_livrees(produit, "Base", index)
        if forme == "trimestres+peak":
            h_peak = heures_livrees(produit, "Peak", index)
            b = part * v / (h_base + (ratio_peak - 1) * h_peak)
            mix[(produit, "Base")] = b
            mix[(produit, "Peak")] = b * (ratio_peak - 1)
        else:
            mix[(produit, "Base")] = part * v / h_base
    return mix
```

- **cal** — un seul ruban : $MW = \text{part} \times \text{volume annuel} / 8760$. La puissance livrée est la même en janvier et en août, alors que la consommation ne l'est pas.
- **trimestres** — un ruban par trimestre, dimensionné sur le volume **de ce trimestre**. La couverture suit la saisonnalité.
- **trimestres+peak** — on ajoute un bloc Peak tel que $(\text{Base} + \text{Peak}) / \text{Base}$ égale le ratio de forme peak/off-peak de notre charge (1,32). La couverture suit alors aussi la forme intra-journalière.

Pour ce dernier cas, le système à résoudre est : $B \cdot H_{\text{total}} + P \cdot H_{\text{peak}} = \text{part} \times V$ et $(B+P)/B = \text{ratio}$, d'où $P = B(\text{ratio}-1)$ puis $B = \text{part} \cdot V / (H_{\text{total}} + (\text{ratio}-1) \cdot H_{\text{peak}})$. C'est exactement la ligne du code.

Le résultat, à couverture identique (70 %, soit 375 639 MWh)

Mix	Produits	Coût	€/MWh
cal	1	31,29 M€	83,31
trimestres	4	32,89 M€	87,55
trimestres + peak	8	33,47 M€	89,11

Le détail du strip trimestriel aux prix du T1 :

Produit	MW	Heures	MWh	Prix T1	Coût
Q1-27 Base	49,58	2 159	107 054	139,00	14,88 M€
Q2-27 Base	37,91	2 184	82 805	45,18	3,74 M€
Q3-27 Base	36,47	2 208	80 517	61,09	4,92 M€
Q4-27 Base	47,65	2 209	105 263	88,78	9,35 M€

Deux lectures à ne pas rater

Suivre la forme coûte 4,24 €/MWh. Le ruban plat est le moins cher parce que le marché price une forte prime d'hiver (Q1-27 à 139 € contre 83,31 pour le Cal). Mais avec un ruban plat on est sous-couvert en janvier et sur-couvert en juillet : on reste exposé au spot exactement quand il est cher. C'est l'arbitrage, et le \$3.4 demande qu'il soit **argumenté**, pas tranché par défaut.

En Q2 et Q3, le Peak est moins cher que le Base (35,04 contre 42,47 ; 48,52 contre 56,20). C'est la cannibalisation solaire : le photovoltaïque écrase les prix de milieu de journée. Comme nos profils pointent le soir (RES1 à 19h30), le Peak nous sert mal **même bon marché**. Rester en Base devient un choix justifié au lieu d'un choix par défaut.

Ce qu'il faut savoir dire

- Qu'un pourcentage sans MW par produit ne répond pas à la question posée.

- Que la couverture totale ne doit **jamais dépasser 100 %** du volume prévisionnel — le code affiche les deux ratios, sur la prévision et sur le réalisé.
- Pourquoi acheter du Peak d'été serait une erreur malgré son prix.

Étape 12 — Les marges et le choix de stratégie

La règle de décision n'est pas « la meilleure espérance », c'est **la meilleure espérance parmi celles dont le pire cas tient le plancher**. Une maximisation sous contrainte, pas une optimisation libre.

Les paramètres, imposés à toutes les équipes

Paramètre	Valeur	Dans le code
Prix de vente, composante énergie	95 €/MWh	PRIX_VENTE
Marge cible	10 €/MWh	MARGE_CIBLE
Plancher de risque (pire cas minimum)	5 €/MWh	PLANCHER_RISQUE
Prime de risque forward	3 %	PRIME_DE_RISQUE

Aucun des quatre n'est une décision d'équipe : ils viennent du §2.2.

Le calcul de la marge

Le cours la pose en mots, session 4 page 24 :

Margin = Sales revenue – Cost of hedges – Cost of the residual volume at spot prices

avec, sous le dernier terme, l'annotation *bought or sold back*. L'exercice corrigé de la session 3 en donne une seconde écriture, algébriquement identique : $\text{marge} = (\text{prix de vente} - \text{coût moyen de l'énergie}) \times \text{volume}$. Développée terme à terme, elle devient :

$$\text{marge} = P_{\{\text{vente}\}} \times V - C_{\{T1\}} - C_{\{T2\}} - \sum_h (\text{charge}_h - \text{livré}_h) \times \text{spot}_h$$

```
def marge(scenario, h1, h2, volumes_q, alpha=None, forme=None, prix_vente=PRIX_VENTE):
    v_ref = scenario["energie_normale"]          # le 100 % des ratios
    forward_t2 = sc.forward_t2_anticipe(scenario["niveau"], alpha)

    mix_t1 = construire_mix(h1, volumes_q, index, forme)
    livre_t1 = livraison(mix_t1, index)
    mw_t2 = h2 * v_ref / len(index)              # tranche T2 : ruban, prix encore inconnu
    residuel = courbe["charge_MW"] - livre_t1 - mw_t2

    cout = (cout_mix(mix_t1, index, "T1") + h2 * v_ref * forward_t2
            + float((residuel * courbe["spot_EUR_MWh"]).sum()))
    marge_eur = prix_vente * scenario["energie_scenario"] - cout
```

Trois choses s'y jouent :

- la **tranche T1** est un vrai mix de produits, aux vrais prix EEX du 22/09 ;
- la **tranche T2** est un ruban plat valorisé au forward *anticipé* : au T1 on ne connaît ni le prix, ni les produits qu'on achètera ;
- le **résiduel** passe au spot heure par heure, et il peut être négatif : s'il l'est, on **revend**. Le §3.5 l'exige nommément (*bought or sold back at the scenario's spot prices*).

Pourquoi la somme est à l'intérieur, et ce que coûterait l'inverse

L'exercice de la session 3 simplifie délibérément : *any volume left uncovered is bought at the average spot price of the year*. Un seul prix pour toute l'année, pour que l'arithmétique reste faisable à la main. Appliquée à notre portefeuille, cette simplification donnerait :

Scénario	Marge horaire	Marge au spot moyen	Écart
2021	4,02	9,24	+5,22
2023	6,30	9,68	+3,38
2024	5,72	9,77	+4,04
choc +20 %	-0,24	6,02	+6,26

La marge serait surestimée de 3,4 à 6,3 €/MWh, et surtout **les quatre scénarios franchiraient le plancher de 5 €/MWh**. Le modèle conclurait que tout va bien.

La raison tient en une phrase : on achète le manque **quand le prix est haut** (janvier, 19 h) et on revend l'excédent **quand il est bas** (août, 4 h du matin). Prendre un prix moyen annuel revient à supposer qu'on achète indifféremment à n'importe quelle heure.

C'est exactement ce que démontre la Part 3 de ce même exercice : risque de volume seul 75 k€, risque de prix seul 0, les deux ensemble 165 k€ — *the 90 of difference cannot be attributed to either one: it comes from their correlation*. D'où l'exigence du §3.5 : **hour by hour or day by day**.

Pourquoi un quatrième scénario de choc

Il faut l'assumer franchement : par construction $\text{Level} = \text{forward} \times (1 - 3\%)$, donc **l'espérance du spot est toujours sous le forward**. Un modèle qui ne contient que des scénarios météo conclut mécaniquement « ne jamais se couvrir ». Ce n'est pas un résultat, c'est un artéfact de la méthode.

Le §3.2 prévoit exactement le remède : *a documented sensitivity such as a price-level shock justified by fundamentals rather than by weather*.

```
CHOC_FONDAMENTAL = 0.20    # 0.0 pour desactiver le scenario de choc
PROBA_CHOC = 0.15         # probabilite du choc (queue haute a 1 sigma)
```

Le choc reprend la météo de l'année la plus froide et ancre son Level sur $\text{forward} \times 1,20$. Le +20 % est la volatilité annuelle CRE, le 15 % correspond à la queue haute à 1 sigma. **Ces deux valeurs sont des choix à défendre**, pas des données : c'est le seul endroit du modèle où vous mettez une opinion.

Les résultats

Stratégie	2021	2023	2024	choc +20 %	Espérance	Pire cas
Notre 70/20/10	4,02	6,30	5,72	-0,24	4,51	-0,24
Variante 50/25/25	4,91	7,88	7,07	-2,26	5,29	-2,26
B - peu couverte	5,20	8,33	7,38	-2,42	5,56	-2,42
C - totalement couverte	4,94	7,79	6,78	-1,07	5,37	-1,07



La figure de `comparaison_scenarios.py` : trois panneaux, quatre scénarios. Les prix spot journaliers, les charges journalières moyennes, et le coût total au spot de chacun.

Le panneau du bas donne la fourchette dans laquelle toute la décision se joue : de **43,5 M€** (scénario 2023) à **56,2 M€** (choc +20 %), soit près de 13 M€ d'écart sur un portefeuille de 537 GWh. Entre les trois années météo seules, l'écart n'est que de 3,4 M€. Autrement dit, c'est le scénario de choc — et lui seul — qui porte le risque auquel la couverture répond ; sans lui, les six stratégies du tableau ci-dessus se tiendraient dans un mouchoir de poche.

Le panneau du milieu contient la démonstration la plus utile de tout le document : **de mai à octobre, les quatre courbes de charge sont confondues**. C'est $g = 0$ hors période de chauffe — sans thermosensibilité, la météo ne change rien au volume, et les scénarios ne se séparent qu'entre novembre et avril. La courbe du scénario 2021 est d'ailleurs invisible : elle est recouverte par celle du choc, qui réutilise la météo de l'année la plus froide et ne modifie que le niveau de prix, jamais le volume.

La relation est **monotone** : plus on couvre, meilleur est le pire cas et plus faible est l'espérance. Trente et une combinaisons ont été balayées ; **aucune ne tient le plancher de 5 €/MWh**. Notre 70/20/10 est celle qui protège le mieux.

Toutes les stratégies testées

A, B, C et D ne sont pas de nous : ce sont les quatre exemples du prof, session 4 page 27, repris avec ses noms et ses ratios. Les guidelines réclament « au moins deux » stratégies suffisamment différentes, en citant 40/40 contre 50/40 contre 40/60 — soit littéralement B, A et C.

Au-delà des six du tableau, le code a balayé 31 combinaisons (T1 de 0 à 100 % par pas de 10, avec complément à 90 % puis à 100 %). Les plus parlantes :

	Espérance	Pire cas
100 / 0 / 0	3,46	1,93 ← meilleure protection
90 / 10 / 0	3,78	1,43
90 / 5 / 5	3,83	1,09
80 / 10 / 10	4,19	0,26
70 / 30 / 0	4,41	0,43
60 / 30 / 10	4,83	−0,74
50 / 50 / 0	5,05	−0,57
0 / 0 / 100	7,60 ← meilleure espérance	−9,84

La relation est **strictement monotone** sur les 31, et aucune ne tient le plancher.

Trois autres dimensions ont été testées : le **mix de produits** à couverture égale (cal 83,31 · trimestres 87,55 · trimestres+peak 89,11 €/MWh), les **sept options du T2** (ne rien acheter, compléter à 85 / 90 / 100 %, chacune en Cal et en trimestres) et les quatre **tests de sensibilité** (plancher, b , α , mix).

Le point à retenir pour l'oral : notre 70/20/10 n'a pas la meilleure espérance, mais **le meilleur pire cas des six**, et ce classement ne bouge sur aucun des quatre tests.

Le coût de la protection

Le §3.5 demande de chiffrer ce qu'on paie pour se protéger. Par rapport à la stratégie la moins couverte :

	Espérance cédée	Pire cas gagné	Prix de la protection
Notre 70/20/10	0,78	2,02	0,39

On cède 0,39 €/MWh d'espérance pour chaque €/MWh de pire cas gagné. C'est la phrase qui fait passer l'étape 5 en Outstanding.

La sensibilité, balayée automatiquement

Le code teste quatre axes à chaque exécution : le plancher (0 à 10), la pente b (minimum annuel, mutualisée, maximum annuel), α (0,2 / 0,5 / 0,6) et le mix (cal / trimestres / +peak). **Le classement des stratégies ne change sur aucun des quatre.** Une décision robuste est un résultat en soi — annoncez-le.

La conclusion, et elle est honnête

À 95 €/MWh imposés, le portefeuille **ne finance pas** une marge protégée de 5 €/MWh. Il faudrait vendre à 98,07 €/MWh avec une couverture totale, ou 100,24 avec notre 70/20/10. Ce n'est pas un bug du modèle : c'est la réponse à la question que pose le §3.5, *what would change your answer if the floor were higher or lower?*

Ce qu'il faut savoir dire

- La règle est **max espérance sous contrainte de pire cas**, et dans cet ordre.
- Pourquoi le scénario de choc est indispensable, et que ses deux paramètres sont vos choix.
- Que le plancher n'est pas atteint, **pourquoi**, et à quel prix de vente il le serait.

Étape 13 — Le travail au T2

La règle d'or du §4 : **les achats du T1 sont un fait, pas une décision**. Le doc prévient même que re-optimiser le T1 avec le recul fait perdre le crédit des deux dates.

```
STRATEGIE_T1 = "Notre strategie 70/20/10"
mix_t1 = cp.construire_mix(cp.STRATEGIES[STRATEGIE_T1][0], volumes_q_t1, index)
```

Le mix T1 entre en dur dans tous les calculs qui suivent et n'est jamais rediscuté.

4.1 — Anticipation contre réalité

Scénario	Forward T2 anticipé	Réel	Erreur	Erreur / sigma
2021	82,02	73,70	-8,32	-1,14
2023	80,66	73,70	-6,96	-0,95
2024	80,39	73,70	-6,69	-0,92
choc +20 %	90,09	73,70	-16,39	-2,25

Le prix observé tombe **hors** de la fourchette anticipée [80,39 ; 82,02]. Les erreurs restent sous 1,2 sigma pour les scénarios météo : le mouvement était banal pour un marché, inatteignable pour un modèle de température.

4.2 — Réviser les probabilités

Le §4.2 est explicite : garder 50/50 est accepté si c'est justifié, mais *what is not accepted is changing the probabilities without a reason, or choosing them so that the strategy you prefer wins*. Il faut donc une **règle mécanique, posée avant de regarder le résultat**.

Celle du code est une mise à jour bayésienne : postérieure $\propto$ antérieure $\times$ vraisemblance du prix observé.

```
def probabilités_revisées(scenarios, priors, alpha=None, forward_observed=None):
    sigma = sc.sigma_t2()
    poids = {an: priors[an] * float(np.exp(
        -0.5 * ((sc.forward_t2_anticipe(s["niveau"], alpha) - observe) / sigma) ** 2))
        for an, s in scenarios.items()}
    total = sum(poids.values())
    return {an: p / total for an, p in poids.items()}
```

La vraisemblance est gaussienne, d'écart-type celui de la CRE. Un scénario dont le forward anticipé tombe loin du prix observé devient moins crédible, mécaniquement.

Scénario	Au T1	Au T2
2021	28,3 %	28,1 %
2023	28,3 %	34,2 %
2024	28,3 %	35,4 %
choc +20 %	15,0 %	2,3 %

Le choc s'effondre : le marché a baissé de 11,5 %, un choc à la hausse devient peu crédible. La révision est motivée par **une information nouvelle**, le prix — pas par la stratégie qu'on préfère.

4.3 — Volume et ratio réellement atteint

Prévision au T1	536 627 MWh
Prévision au T2	501 962 MWh (-6,46 %)
Acheté au T1	375 639 MWh, soit 70,0 % de l'ancienne prévision
Ratio réellement atteint	74,8 % de la nouvelle

Le portefeuille a rétréci, donc on est **sur-couvert de 4,8 points** par rapport au plan, sans avoir rien fait. La marge de manœuvre jusqu'à 100 % tombe de 30 % à 25,2 %. C'est le risque de volume du cours, en une ligne.

Le mark-to-market de la position T1

```
for (p, n), mw in sorted(mix.items()):
    h = cp.heures_livrees(p, n, index)
    t1, t2 = sc.prix_eex(p, n, "T1"), sc.prix_eex(p, n, "T2")
    lignes.append({... "MtM MEUR": mw * h * (t2 - t1) / 1e6})
```

Produit	MWh	Prix T1	Prix T2	Écart	MtM
Q1-27 Base	107 054	139,00	119,54	-19,46	-2,08 M€
Q2-27 Base	82 805	45,18	42,47	-2,71	-0,22 M€
Q3-27 Base	80 517	61,09	56,20	-4,89	-0,39 M€
Q4-27 Base	105 263	88,78	77,24	-11,54	-1,21 M€
Total	375 639				-3,92 M€

Soit **-10,43 €/MWh couvert**. Ce n'est pas une erreur : on a acheté une assurance qui ne s'est pas déclenchée. La formuler ainsi est plus solide que de la passer sous silence.

4.4 — Décider la part restante

Sept options testées (quatre cibles × deux formes), T1 figé, Levels réancrés sur 73,70, nouvelles probabilités, nouveau volume.

Option	Achat T2	Couverture	Espérance	Pire cas	Admissible
Ne rien acheter de plus	0 %	74,8 %	8,24	2,49	non
Compléter à 85 % - cal	10,2 %	85 %	7,81	3,71	non
Compléter à 90 % - cal	15,2 %	90 %	7,60	4,31	non
Compléter à 100 % - cal	25,2 %	100 %	7,14	5,51	oui

Décision : acheter 14,42 MW de Cal-27 Base à 73,70 €/MWh, soit 126 323 MWh pour 9,31 M€. C'est la seule option dont le pire cas tient le plancher.

Notez l'inversion par rapport au T1 : **au T2, le plancher devient atteignable**, parce que le marché a baissé de 11,5 % et que la part non couverte s'achète beaucoup moins cher qu'au 22 septembre.

Ce qu'il faut savoir dire

- Que le T1 est **figé**, et pourquoi le doc en fait une faute éliminatoire.
- Que la règle de révision des probabilités a été posée **avant** d'en voir l'effet.
- Que le mark-to-market négatif est le prix d'une assurance, pas une erreur de jugement.

Étape 14 — La position ouverte du 1er janvier 2027

Une couverture est un **engagement de livraison, pas un stock** : un Cal-27 Base de 14 MW livre 14 MW à chaque heure de l'année, y compris à 4 h du matin le 1er janvier quand on n'en consomme que 42. L'écart se solde au day-ahead.

$$\text{position ouverte}_h = \text{charge prévue}_h - \text{couverture livrée}_h$$

Positive, on est **short** : il faut acheter. Négative, on est **long** : il faut revendre.

Les trois pièges du 1er janvier 2027

1. **C'est un vendredi.** Dans le calendrier EEX c'est donc un jour ouvré : les produits Peak livrent.
2. **C'est un jour férié.** Les clients consomment comme un dimanche. Le §5.2 l'écrit noir sur blanc : *using a holiday profile and not an ordinary Friday*.
3. **Les deux à la fois.** Qui a acheté du Peak reçoit de la puissance de 8 h à 20 h un jour où ses clients sont chez eux : il se retrouve **long en pleine journée**, à revendre au pire moment.

Notre portefeuille n'a que du Base, donc le troisième piège ne nous touche pas. **Revendiquez-le comme un choix**, pas comme une chance : il découle de l'analyse peak/off-peak de l'étape 11.

Pourquoi le profil férié est déjà là

Rien de spécial à coder : le travail a été fait à l'étape 6. `_type_jour()` renvoie 7 pour les fériés, donc `appariement_jours()` est allé chercher un **jour férié de 2025** pour alimenter le 1er janvier 2027. La décision prise soixante lignes plus tôt dans `calendrier.py` paie ici.

De même, la conversion des produits en MW livrés réutilise telle quelle la fonction de l'étape 11, simplement restreinte au jour :

```
jour = index[np.asarray(index.date) == pd.Timestamp(JOUR_SPOT).date()]
livre = cp.livraison(mix_final, jour)
pos["position normale MW"] = pos["charge normale MW"] - pos["couverture MW"]
pos["sens"] = np.where(pos["position normale MW"] > 0, "SHORT (acheter)", "LONG (revendre)")
```

`mix_final` est la fusion du mix T1 et du mix T2.

Le tableau

Couverture plate de **64,01 MW** : 49,58 MW de Q1-27 achetés au T1, plus 14,42 MW de Cal-27 achetés au T2. La charge, elle, va de 42,21 à 98,71 MW.

Heure	Charge (MW)	Couverture (MW)	Position (MW)	Sens
00:00-01:00	54,26	64,01	-9,74	long
03:00-04:00	43,22	64,01	-20,78	long
05:00-06:00	42,21	64,01	-21,80	long (maximum)
08:00-09:00	62,40	64,01	-1,61	long (bascule)
09:00-10:00	72,71	64,01	+8,71	short
12:00-13:00	91,39	64,01	+27,39	short
15:00-16:00	72,32	64,01	+8,32	short (creux)
19:00-20:00	98,71	64,01	+34,70	short (maximum)
22:00-23:00	69,96	64,01	+5,95	short
23:00-00:00	56,39	64,01	-7,62	long

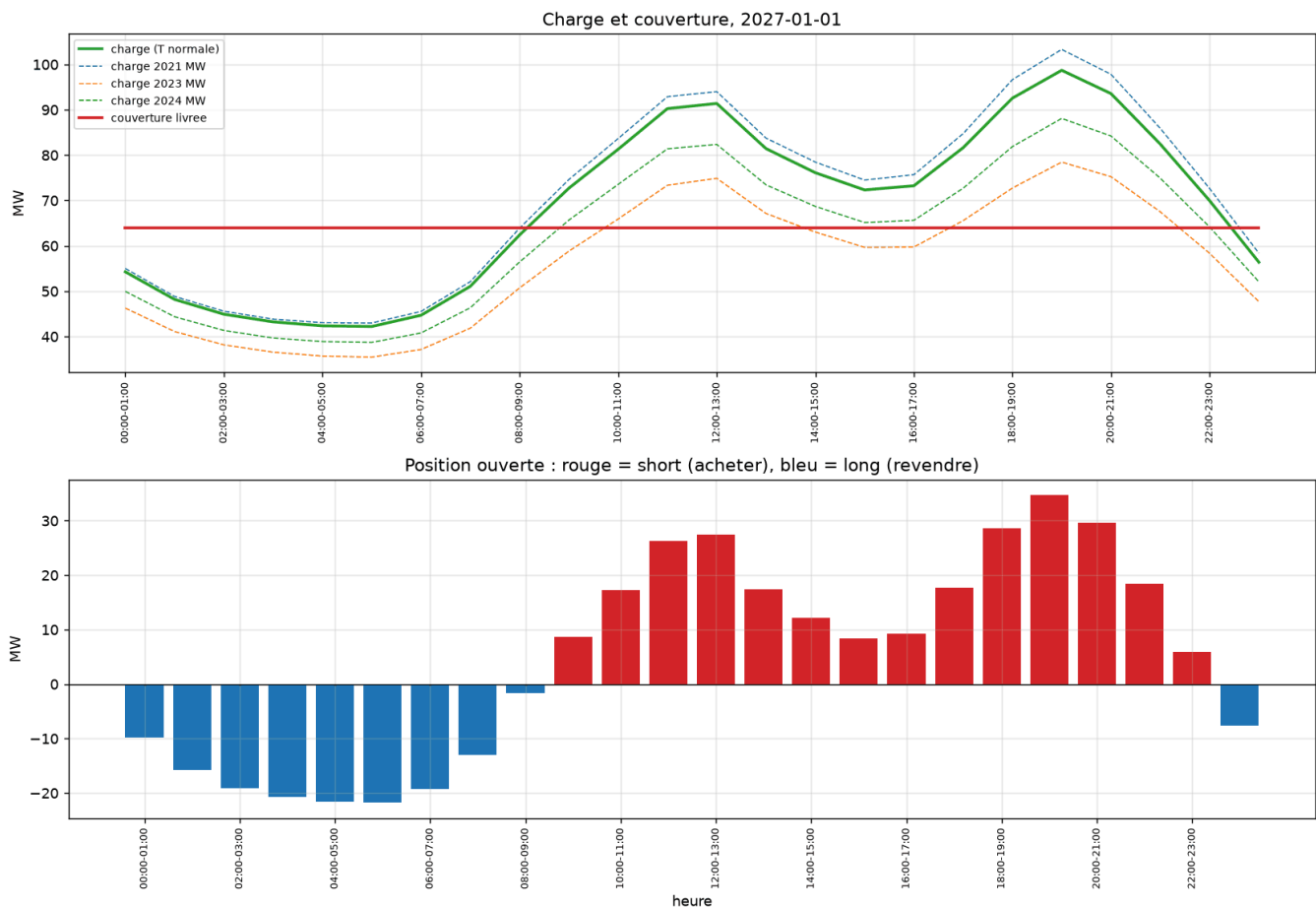
Au total : 261,6 MWh à acheter et 150,4 MWh à revendre sur la journée.

Comment lire cette courbe

Le profil est typiquement férié : creux de nuit très bas, pas de pointe du matin, un double sommet à midi (91 MW) et surtout le soir (99 MW à 19 h). Face à lui, un ruban plat à 64 MW.

Résultat : **long de minuit à 9 h, short de 9 h à 23 h, long à nouveau en fin de soirée**. C'est le défaut structurel du produit Base — il ne connaît pas les heures. Un ruban dimensionné sur une moyenne annuelle sur-couvre les creux et sous-couvre les pointes, et c'est exactement ce que la position ouverte rend visible.

Le fichier `position_ouverte_20270101.csv` contient le tableau complet avec une colonne de position par scénario. C'est lui qu'il faut ouvrir dans Excel pour la session 5 ; `decision_t2.py` trace déjà le graphique en barres (rouge au-dessus de zéro pour les heures à acheter, bleu en dessous pour celles à revendre).



La voilà. En haut : la charge à température normale (trait vert épais), les trois scénarios météo (pointillés), et la **couverture livrée à 64 MW plats** (trait rouge) — un ruban de forwards ne sait pas faire autre chose qu'une ligne droite. En bas : la différence des deux, heure par heure.

Deux choses à lire. **La position change de signe deux fois** : longue (bleu) de minuit à 8h et de nouveau à 23h, courte (rouge) de 9h à 22h. Ce n'est pas un défaut de dimensionnement : un produit plat posé sur une courbe en cloche est forcément trop grand la nuit et trop petit le soir. **L'amplitude est asymétrique** : jusqu'à -22 MW à revendre vers 5h, mais +35 MW à acheter vers 19h. On revend au creux, donc au prix bas, et on achète à la pointe, donc au prix haut. C'est tout le coût résiduel que la marge de l'étape 12 encaisse.

Les pointillés donnent la mesure de ce qui reste incertain : à 19h, selon l'année météo retenue, la charge va de 79 MW (scénario 2023) à 104 MW (scénario 2021). Vingt-cinq MW d'écart sur une seule heure, qu'aucun forward ne couvre — c'est ce résidu-là qui passera par le spot.

Ce qu'il faut savoir dire

- Qu'une couverture **livre**, elle ne se stocke pas : c'est pour ça qu'on peut être long.
- Le piège vendredi férié du Peak, même si on n'en a pas acheté.
- Que la forme de la position ouverte est la conséquence directe du choix Base, pas un hasard.

Étape 15 — Le T3 et l'ordre day-ahead

Le 31 décembre, trois choses changent d'un coup : le marché a rebondi, le portefeuille s'est reconstitué, et **une centrale solaire de 80 MWc apparaît** — elle n'existait dans aucun document précédent.

	T1 (22/09)	T2 (01/12)	T3 (31/12)
Forward Cal-27 Base	83,31	73,70	78,37
Q1-27 Base	139,00	119,54	133,88
Clients RES1	200 000	188 200	191 531
Volume 2027	536 627	501 962	512 820 MWh

Le Q1-27 reprend 12 % en un mois : le marché anticipe un janvier froid, ce que la prévision de température confirmera. Tous les produits font le même V entre les trois jalons : baisse jusqu'au 1er décembre, remontée au 31. Mais **l'amplitude est très inégale** — le Q1-27 perd puis reprend une vingtaine d'euros quand le Q2-27 bouge à peine de trois. C'est l'hiver qui porte toute la volatilité, et c'est précisément là que notre couverture était la plus faible.

Objectif 1 — rééquilibrer la couverture

Les achats du T1 et du T2 sont un **fait**, pas une décision : dans `spot_t3.py` ils sont une constante `POSITION`, pas un calcul. Valorisés aux prix du 31/12 :

Produit	MW	Acheté au	MWh	Prix payé	Prix T3	MtM
Q1-27 Base	49,58	T1	107 043	139,00	133,88	−548 k€
Q2-27 Base	37,91	T1	82 795	45,18	43,32	−154 k€
Q3-27 Base	36,47	T1	80 526	61,09	56,76	−349 k€
Q4-27 Base	47,65	T1	105 259	88,78	80,33	−889 k€
Cal-27 Base	14,42	T2	126 319	73,70	78,37	+590 k€
Total			501 942	84,06		−1,35 M€

L'achat du T2 était le bon appel : acheté au creux du marché, il est le seul en gain. Et le mark-to-market global s'est redressé de −3,92 M€ au T2 à −1,35 M€.

Le ratio global tombe à 97,9 %, mais la moyenne cache l'essentiel :

	Besoin T3	Couvert	Ratio	Écart
Q1 (hiver)	146 113	138 176	94,6 %	short 7 937 MWh
Q2	113 058	114 289	101,1 %	long 1 231 MWh
Q3 (été)	109 950	112 365	102,2 %	long 2 415 MWh
Q4 (hiver)	143 699	137 113	95,4 %	short 6 586 MWh

On est short en hiver et long en été en même temps. C'est le défaut du ruban Cal plat acheté au T2, qui livre autant en janvier qu'en juillet. Le rééquilibrage s'impose donc **par trimestre**, pas par un bout de Cal supplémentaire :

ACHETER	3,68 MW de Q1-27 Base a 133,88	->	+1 063 kEUR
VENDRE	0,56 MW de Q2-27 Base a 43,32	->	-53 kEUR
VENDRE	1,09 MW de Q3-27 Base a 56,76	->	-137 kEUR
ACHETER	2,98 MW de Q4-27 Base a 80,33	->	+529 kEUR

	solde		+1,40 MEUR

Objectif 2 — l'ordre day-ahead du 1er janvier

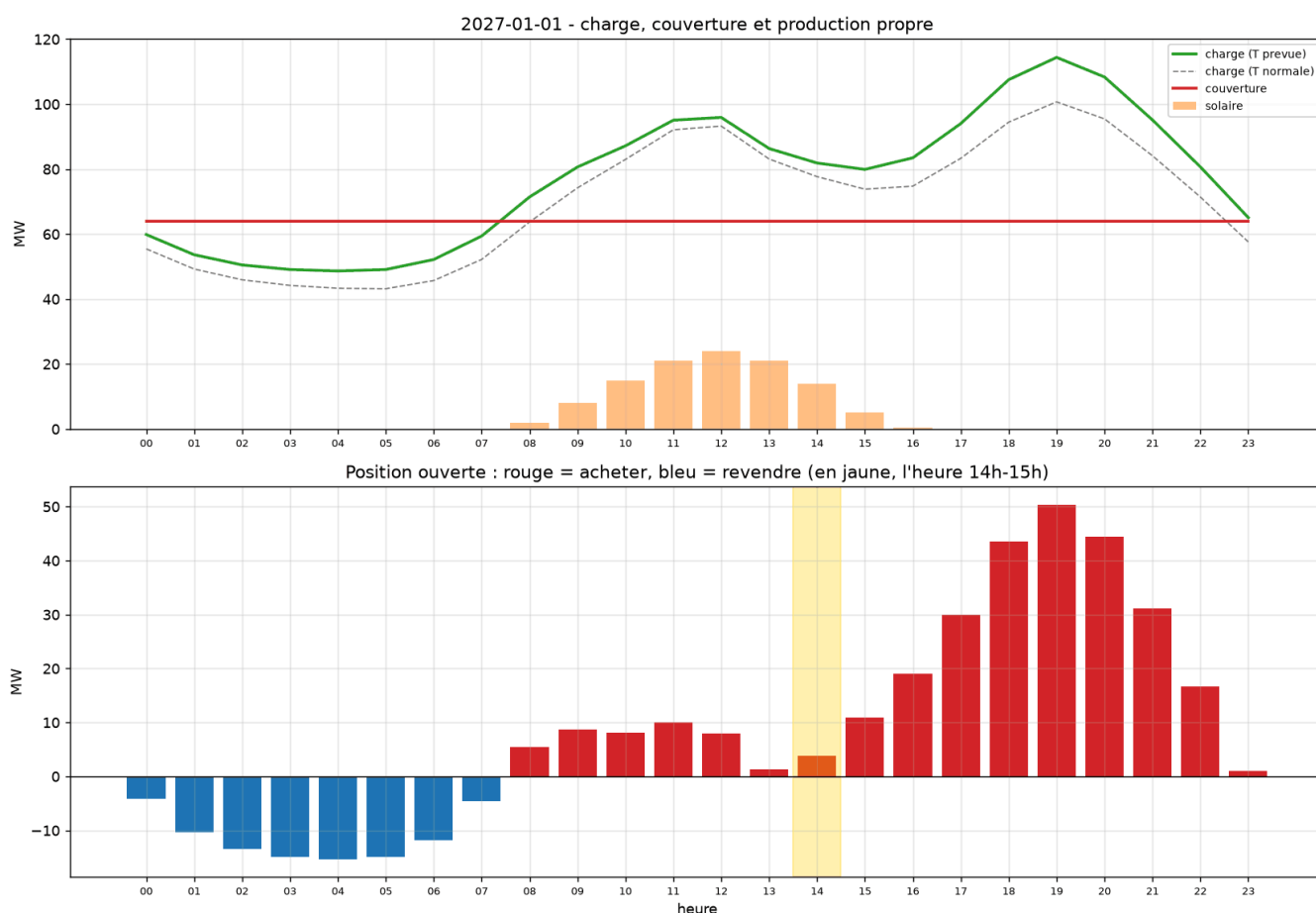
La définition vient des takeaways de la session 5 :

Your spot exposure is the open position: **load – hedges – own generation**, for each 15-minute MTU.

Deux termes nouveaux par rapport à l'étape 14 : la **température prévue** (+0,44 °C de moyenne contre 4,79 °C de normale, donc `CM` à 1,09 pour RES1 et 1,14 pour PRO1) et la **production solaire**.

Heure	Charge	Solaire	Couverture	Position
04:00	48,65	0	64,00	−15,35 long
12:00	95,92	24,0	64,00	+7,92
14:00	81,89	14,0	64,00	+3,89
15:00	79,91	5,0	64,00	+10,91
19:00	114,39	0	64,00	+50,39 short

Sur la journée : 1 849,5 MWh de charge contre 1 682,1 à température normale, 110,5 MWh de solaire, **292,6 MWh à acheter et 89,6 à revendre**.



La même journée que l'étape 14, trois mois plus tard, avec deux informations de plus. C'est la figure de `spot_t3.py`. En haut : la charge à température prévue (vert), la charge à température normale (pointillé gris), la couverture à 64 MW (rouge) et la production solaire (barres orange). En bas : la position ouverte heure par heure, l'heure 14h-15h de l'objectif 2 surlignée en jaune.

L'écart entre le trait vert et le pointillé gris, c'est la prévision météo — **+0,44 °C annoncés contre 4,79 °C de normale**, donc un CM autour de 1,09, donc 4 à 14 MW de charge en plus selon l'heure. Les barres orange, c'est la centrale de 80 MWc : elle ne produit qu'entre 8h et 16h, culmine à 24 MW vers midi, et **ne touche jamais la pointe du soir**. Une centrale solaire ne couvre pas un portefeuille résidentiel d'hiver, elle déplace seulement le problème de midi vers 19h.

Le panneau du bas le montre directement : la journée reste **longue de minuit à 7h** (jusqu'à 15 MW à revendre), passe short à partir de 8h, puis se creuse à 13h-14h — c'est le solaire qui comble presque tout le besoin de milieu de journée — avant le mur du soir à **+50 MW à 19h**. À 14h, la barre surlignée vaut **+3,89 MW** : c'est le chiffre de l'objectif 2.

Pour l'heure demandée, la courbe d'ordre découle d'un principe de la session 5 : *own generation is offered to the market, **never reserved for your customers** — you dispatch against the price.*

```
prix < 0 EUR/MWh : on efface le solaire -> ACHETER 17,89 MWh
prix >= 0 EUR/MWh : le solaire est vendu -> ACHETER 3,89 MWh
```

Les 3,89 MWh sont un volume **subi** : ordre sans limite de prix, parce que ne pas être servi coûte plus cher que n'importe quel prix de clearing. Le solaire, lui, porte une limite à son coût marginal (zéro) : en dessous, on préfère l'effacer.

La réserve qu'il faut poser

Si la centrale de 80 MWc appartient au portefeuille **pour l'année entière**, elle produit de l'ordre de 90 à 100 GWh, soit 18 à 20 % du besoin. L'objectif 1 s'inverserait alors complètement : on ne serait plus sous-couvert à 97,9 %, mais largement sur-couvert, et il faudrait vendre partout au lieu d'acheter l'hiver.

Le fichier ne donne la production que pour le 1er janvier. La constante `SOLAIRE_ANNUEL_MWH` est donc laissée à `None` dans le code : la renseigner suffit à faire basculer tout le raisonnement de l'objectif 1 vers un besoin net. À trancher avec le prof avant de figer la réponse.

Même logique pour `PRIX_CLEARING`, laissé à `None` : le *hedge success* ne se calcule qu'une fois le prix spot publié — et le terme lui-même n'est défini nulle part dans les slides.

Les limites à savoir nommer

Trois faiblesses connues. Les annoncer vous-même vaut toujours mieux que de les laisser surgir dans les questions.

1. La structure par terme du Shape ne suit pas le marché

C'est la plus sérieuse. Le Level × Shape normalise la forme historique à une moyenne **annuelle** de 1 et l'ancre sur le forward annuel. La saisonnalité qui en résulte est donc celle de l'année historique, pas celle que le marché price pour 2027.

	Q1 / moyenne	Q4 / moyenne
Marché EEX au T1	1,67	1,07
Scénario 2021	0,49	2,03
Scénario 2023	1,35	0,83
Scénario 2024	1,09	1,50

2021 traîne la crise gazière de son quatrième trimestre, 2023 un premier trimestre cher. Conséquence directe : quand le modèle compare un achat en trimestres (aux vrais prix EEX) à un achat en Cal, il juge les trimestres « trop chers » — mais c'est parce qu'il confronte un prix Q1 **réel** à un spot Q1 tiré au hasard dans l'histoire. **Toute comparaison Cal contre trimestres est donc peu fiable en l'état.**

Un point important pour se défendre : la méthode telle qu'elle est implémentée suit **exactement** la prescription du cours (*normalising each shape to an average of 1, never reusing the historical price level*). Ce n'est pas une erreur de codage, c'est une limite de la méthode prescrite.

Le correctif, si vous voulez aller plus loin : normaliser la forme à l'intérieur de **chaque trimestre** et ancrer chaque trimestre sur son propre prix EEX (119,54 / 42,47 / 56,20 / 77,24 au T2). L'année historique ne fournirait plus que la forme intra-trimestre et intra-journalière, qui est son vrai apport. Une quinzaine de lignes dans `scenario_spot()`.

2. Le r^2 de la régression vaut 0,243

Le volume explique un quart de la variance du prix, pas davantage. Les pentes annuelles vont de 1,93 à 3,62, presque du simple au double. Le code mutualise sur 454 jours pour stabiliser l'estimation et teste la sensibilité — le classement des stratégies ne bouge pas, ce qui est rassurant. Mais **b n'est pas une loi**, c'est une régularité statistique moyenne, et il faut le présenter ainsi.

3. Trois années douces, et un choc dont vous choisissez les paramètres

2021, 2023 et 2024 ont des anomalies de volume comprises entre −1,69 % et −0,05 % : **aucune n'est un vrai hiver froid**. C'est la raison d'être du scénario de choc, mais ses deux paramètres (+20 % de niveau, 15 % de probabilité) sont des choix défendables, pas des mesures.

La meilleure amélioration possible du dossier, si vous avez le temps : **ajouter une vraie année froide** (2012 ou 2017) à la liste `ANNEES`. Un scénario froid documenté vaut mieux qu'un choc synthétique, et le code le prend sans modification — il suffit des deux CSV dans `historique/`.

Ce que le modèle ne fait pas du tout

Deux sujets apparaissent dans le cours mais **ne sont pas exigés** par les guidelines du Market Game :

- la **production photovoltaïque** et la duck curve (session 2) ;
- une génération de scénarios par **séries temporelles ou Monte-Carlo** — le §3.2 la propose comme alternative au calcul de `b`, pas comme obligation, et prévient qu'il vaut mieux maîtriser l'outil qu'on utilise.

Les citer comme prolongements possibles vaut mieux que de faire comme s'ils n'existaient pas.